



รายงานโครงการ

PokéVault Commerce

ระบบจัดการร้านค้าการ์ด Pokémon TCG Pocket

จัดทำโดย

673380292-5 นายศิฆรินทร์ อุปจันทร์

673380272-1 นายธนภูมิ จันทรา

673380294-1 นายสรวิชัย ศาสสนสุพินธุ์

673380066-4 นายสัพพัญญ คำตุ้ม

673380301-0 นายแทนคุณ พันธนิกุล

เสนอ

รศ.ดร.ปัญญาพล หอระตะ

รายวิชา CP353002

Principles of Software Design and Development

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

บทคัดย่อ

PokéVault Commerce เป็นเว็บแอปพลิเคชันสำหรับร้านค้าการ์ด Pokémon TCG Pocket ที่ใช้การสนทนากับลูกค้าและการเทรดในเกมเป็นส่วนหนึ่งของการส่งมอบสินค้า ระบบรวบรวมข้อมูลแค็ตตาล็อกการ์ด บัญชีเกมของร้าน สต็อก คำสั่งซื้อ และความคืบหน้าการเทรดไว้ในแหล่งข้อมูลเดียว เพื่อช่วยให้พนักงานตรวจสอบได้ว่าการ์ดที่ลูกค้าจองอยู่ในบัญชีใด และส่งมอบครบแล้วหรือยัง

โครงการพัฒนาด้วย Java 17 และ Spring Boot 3.4.3 ใช้ Thymeleaf เป็นส่วนติดต่อผู้ใช้ Spring Data JPA สำหรับเข้าถึงฐานข้อมูล และ PostgreSQL ในสภาพแวดล้อมใช้งานจริง การออกแบบแบ่ง Presentation Service Repository และ Domain อย่างชัดเจน ใช้ Enterprise Patterns 6 รายการ และ GoF Behavioral Patterns 3 แบบ ได้แก่ Strategy สำหรับส่วนลด State สำหรับวงจรคำสั่งซื้อ และ Observer สำหรับติดตามสต็อกต่ำ

ฐานข้อมูลมี 8 ตาราง ครอบคลุมความสัมพันธ์หนึ่งต่อหนึ่งแบบเลือกมีโปรไฟล์ได้ และหนึ่งต่อหลาย มีข้อกำหนด Primary Key Foreign Key Unique และ Check ประกอบกับการกำหนด Fetch และ Cascade ใน JPA REST API มี CRUD สำหรับ Cards และ Game Accounts พร้อม Validation การแบ่งหน้า การเรียงลำดับ และเอกสาร OpenAPI ผ่าน Swagger UI แผนภาพในรายงานเรนเดอร์จากต้นฉบับ PlantUML ของโครงการ

ผลทดสอบที่บันทึกเมื่อวันที่ 9 ตุลาคม 2569 มี 373 กรณี ผ่านทั้งหมดโดยไม่มี Failure Error หรือ Skipped ครอบคลุม Unit Test Controller และ Security Test รวมถึง Persistence และ Schema Test บน H2 ผลดังกล่าวไม่ใช้การวัด Coverage 100 เปอร์เซ็นต์ และไม่ได้ทดแทนการทดสอบฐานข้อมูล PostgreSQL หรือการทำงานผ่านเบราว์เซอร์ ระบบมี URL สาธารณะบน Render และใช้ Neon เป็นฐานข้อมูล PostgreSQL

รายงานฉบับนี้อธิบายปัญหา ขอบเขตระบบ หลักการออกแบบ ความสัมพันธ์ระหว่าง Diagram กับโค้ด การออกแบบฐานข้อมูล REST API ส่วนติดต่อผู้ใช้ การทดสอบ การทำงานร่วมกัน และการติดตั้ง เพื่อให้ตรวจสอบการตัดสินใจออกแบบและหลักฐานที่ส่งมอบได้ครบถ้วน

คำสำคัญ

Spring Boot Layered Architecture SOLID Strategy State Observer PostgreSQL Chat Commerce

แหล่งใช้งานและซอร์สโค้ด

เว็บไซต์ <https://pok-vault-commerce.onrender.com>

Repository https://github.com/Bigzzz0/Pok-Vault_Commerce

สารบัญ

บทคัดย่อ	ก
บทที่ 1 บทนำและขอบเขตโครงการ	1
บทที่ 2 โครงสร้างระบบและ SOLID	6
บทที่ 3 Design Patterns และเหตุผลการเลือกใช้	15
บทที่ 4 การทำงานของ Use Cases	26
บทที่ 5 การออกแบบฐานข้อมูล	31
บทที่ 6 REST API และส่วนติดต่อผู้ใช้	37
บทที่ 7 การทดสอบและการส่งมอบ	55
บทที่ 8 ผลงาน ข้อจำกัด และข้อสรุป	64
เอกสารอ้างอิง	65
ภาคผนวก ก Data Dictionary	67
ภาคผนวก ข ผลทดสอบบราย Suite	76
ภาคผนวก ค Diagram ฉบับสรุป	80
ภาคผนวก ง การตรวจสอบตามหัวข้อประเมิน	84

สารบัญภาพ

ภาพที่ 1 Use Case Diagram ของระบบ	3
ภาพที่ 2 Domain Model และความสัมพันธ์ของข้อมูล	6
ภาพที่ 3 Class Diagram ของ Card Catalog	7
ภาพที่ 4 Layered Architecture และเส้นทาง Request	8
ภาพที่ 5 Class Diagram แสดง Strategy State และ Observer	17
ภาพที่ 6 Class Diagram ของ Discount Strategy	18
ภาพที่ 7 Class Diagram ของ Order State Pattern	20
ภาพที่ 8 State Diagram ของคำสั่งซื้อ	21
ภาพที่ 9 State Diagram ของการส่งมอบรายการ	21
ภาพที่ 10 Sequence Diagram ของ OrderPlacedEvent และ LowStockObserver	23
ภาพที่ 11 Sequence Diagram การสั่งจองการ์ด	26
ภาพที่ 12 Sequence Diagram การบันทึกการ์ดที่เปิดช่องได้	27
ภาพที่ 13 Sequence Diagram การรับเงิน จับคู่ และส่งมอบ	28
ภาพที่ 14 Activity Diagram การสร้างคำสั่งซื้อ	29
ภาพที่ 15 Activity Diagram การส่งมอบและตรวจครบทุก Item	30
ภาพที่ 16 ER Diagram ภาพรวมทั้ง 8 ตาราง	31
ภาพที่ 17 ER Diagram กลุ่มการ์ด บัญชีเกม และสต็อก	32
ภาพที่ 18 ER Diagram กลุ่มผู้ใช้และคำสั่งซื้อ	33
ภาพที่ 19 Swagger UI จากบริการที่ Deploy บน Render	44

สารบัญภาพต่อ

ภาพที่ 20 หน้าแรกของเว็บที่ Deploy แล้ว	46
ภาพที่ 21 หน้าแกลเลอรีพร้อมการค้นหาและตัวกรอง	47
ภาพที่ 22 หน้าต่างรายละเอียดการ์ดและจำนวนคงเหลือ	47
ภาพที่ 23 หน้าคลังสินค้าแสดงการ์ด สภาพ จำนวน และราคา	49
ภาพที่ 24 รีวิวการ์ดในหน้าต่างสร้างคำสั่งซื้อ	50
ภาพที่ 25 หน้าจัดการบัญชีเกมและความพร้อมเทรด	51
ภาพที่ 26 หน้าคำสั่งซื้อและสถานะปัจจุบัน	52
ภาพที่ 27 หน้าต่างจัดการเทรดและบัญชีที่ใช้ส่งมอบ	52
ภาพที่ 28 Component Diagram ของระบบ	60
ภาพที่ 29 Deployment Diagram บน Render และ Neon	61
ภาพที่ 30 Sequence ฉบับสรุปการจอง	80
ภาพที่ 31 Sequence ฉบับสรุปการบันทึกเปิดซอง	80
ภาพที่ 32 Sequence ฉบับสรุปการเทรด	81
ภาพที่ 33 Activity ฉบับสรุปการจอง	81
ภาพที่ 34 Use Case ฉบับสรุปของระบบ	82
ภาพที่ 35 Activity ฉบับสรุปการส่งมอบ	82
ภาพที่ 36 Use Case ของ Visitor และ CUSTOMER	83
ภาพที่ 37 Use Case ของ STAFF และ ADMIN	83

สารบัญตาราง

ตารางที่ 1 กลุ่มผู้ใช้และการใช้งาน	1
ตารางที่ 2 ความต้องการเชิงหน้าที่	2
ตารางที่ 3 หน้าที่ของแต่ละ Layer	8
ตารางที่ 4 หลักฐาน Single Responsibility	10
ตารางที่ 5 ตัวอย่าง Contract ของ DiscountStrategy	11
ตารางที่ 6 การแบ่ง Service Interfaces	12
ตารางที่ 7 Enterprise Patterns กลุ่มโครงสร้าง	15
ตารางที่ 8 Enterprise Patterns กลุ่มบริการและข้อมูล	16
ตารางที่ 9 ตำแหน่งและบทบาทของ GoF Patterns	17
ตารางที่ 10 ตัวอย่างการคำนวณราคาเป็นบาท	19
ตารางที่ 11 กฎ SQL และ JPA ที่ต้องอ่านแยกกัน	34
ตารางที่ 12 Constraints ที่ใช้รักษาความถูกต้อง	35
ตารางที่ 13 HTTP Methods และ Status Codes	37
ตารางที่ 14 Endpoints ของ Card Catalog	38
ตารางที่ 15 Endpoints ของ Game Account Vault	39
ตารางที่ 16 Endpoints ในการจองและส่งมอบ	40
ตารางที่ 17 สัญญาข้อผิดพลาด	41
ตารางที่ 18 พารามิเตอร์การแบ่งหน้าและตัวกรอง	43
ตารางที่ 19 ขอบเขตสิทธิ์การใช้งาน	45
ตารางที่ 20 การเตรียมไฟล์รูปสำหรับแอป	54

สารบัญตารางต่อ

ตารางที่ 21 ระดับการทดสอบที่มีในโครงการ	55
ตารางที่ 22 หลักฐานผลทดสอบรวม	56
ตารางที่ 23 กรณีทดสอบที่สัมพันธ์กับกฎธุรกิจ	57
ตารางที่ 24 สมาชิกและความรับผิดชอบ	58
ตารางที่ 25 Configuration สำหรับ Deployment	62
ตารางที่ 26 ภาพรวม Data Dictionary	67
ตารางที่ 27 Data Dictionary ของ users	68
ตารางที่ 28 Data Dictionary ของ user_profiles	69
ตารางที่ 29 Data Dictionary ของ card_expansions	70
ตารางที่ 30 Data Dictionary ของ cards	71
ตารางที่ 31 Data Dictionary ของ game_accounts	72
ตารางที่ 32 Data Dictionary ของ card_inventories	73
ตารางที่ 33 Data Dictionary ของ orders	74
ตารางที่ 34 Data Dictionary ของ order_items	75
ตารางที่ 35 ผลทดสอบราย Suite ชุด 1	76
ตารางที่ 36 ผลทดสอบราย Suite ชุด 2	77
ตารางที่ 37 ผลทดสอบราย Suite ชุด 3	78
ตารางที่ 38 ผลทดสอบราย Suite ชุด 4	79
ตารางที่ 39 แผนพื้นฐานตามหัวข้อประเมิน	84

บทที่ 1 บทนำและขอบเขตโครงการ

1.1 ที่มาและปัญหา

การขายการ์ดใน Pokémon TCG Pocket ต้องประสานงานระหว่างข้อมูลหน้าร้าน การรับเงินในแชท และการส่งการ์ดผ่านบัญชีเกมของร้าน การซื้อเดียวกันอาจอยู่หลายบัญชี มีสภาพและราคาต่างกัน หากบันทึกข้อมูลแยกกัน พนักงานอาจรับจองเกินจำนวน เลือกบัญชีที่ไม่พร้อมเทรด หรือปิดคำสั่งซื้อก่อนส่งมอบครบทุกใบ

กลุ่มจึงพัฒนาระบบที่เชื่อมต่อตลาดล็อกเข้ากับสต็อกจริง และเชื่อมคำสั่งซื้อแต่ละรายการกับบัญชีเกมที่ใช้ส่งมอบ พนักงานเห็นขั้นตอนการดำเนินงานในหน้าเดียว ขณะที่ลูกค้าเลือกการ์ดและติดตามคำสั่งซื้อของตนได้ การคำนวณยอดเงินและการเปลี่ยนสถานะถูกควบคุมที่ Service เพื่อให้หน้าเว็บและ REST API ใช้กฎเดียวกัน

1.2 วัตถุประสงค์

- รวบรวมข้อมูลการ์ด ชุดการ์ด บัญชีเกม และสต็อก ให้ตรวจสอบแหล่งเก็บการ์ดได้
- สร้างคำสั่งซื้อโดยหักสต็อกและคำนวณส่วนลดตามระดับสมาชิกจากข้อมูลของระบบ
- ควบคุมวงจรคำสั่งซื้อและการส่งมอบรายการ เพื่อป้องกันการข้ามสถานะ
- พัฒนา REST API และหน้าเว็บที่ทำงานร่วมกัน พร้อมหลักฐาน UML โค้ด และผลทดสอบ
- ประยุกต์ใช้ Layered Architecture SOLID และ Design Patterns พร้อมอธิบายเหตุผลการเลือกใช้

1.3 ผู้ใช้และประโยชน์ที่ได้รับ

ตารางที่ 1 กลุ่มผู้ใช้และการใช้งาน

ผู้ใช้	งานหลัก	ประโยชน์
Visitor	ดูการ์ด ค้นหา สมัครสมาชิก และเข้าสู่ระบบ	เห็นสินค้าและข้อมูลก่อนสั่งจอง
CUSTOMER	สั่งซื้อและดูคำสั่งซื้อของตนเอง	ทราบราคา ส่วนลด และความคืบหน้า
STAFF / ADMIN	จัดการคลัง บัญชีเกม ราคา สมาชิก และเทรด	ตรวจสอบสต็อกและส่งมอบเป็นลำดับ

1.4 ผลงานที่ส่งมอบ

เว็บแอปพลิเคชัน ซอร์สโค้ดใน GitHub REST API และ Swagger ชุด UML และภาพ Diagram เอกสารการออกแบบ รายงานทดสอบ Dockerfile และการตั้งค่า CI พร้อมเว็บไซต์ที่ <https://pok-vault-commerce.onrender.com>

1.5 ขอบเขตและความต้องการของระบบ

ระบบรองรับงานหน้าร้านและงานบริหารคลังของร้านเดียว โดยแบ่งข้อมูลสินค้าออกจากจำนวนคงเหลือในแต่ละบัญชีเกม และเก็บราคา ณ เวลาจองในรายการคำสั่งซื้อ กฎการคำนวณเงินใช้ **BigDecimal** และปิดเศษสองตำแหน่งตามโค้ด

ตารางที่ 2 ความต้องการเชิงหน้าที่

รหัส	ความต้องการเชิงหน้าที่	ส่วนที่ทำงาน
FR01	ดู ค้นหา และกรองการด์ตามชุด ธาตุ ประเภท และความหายาก	Catalog / หน้า cards
FR02	สมัครสมาชิก เข้าสู่ระบบ และกำหนดสิทธิ์ตามบทบาท	Security / Web
FR03	เพิ่ม แก้ไข อ่าน และลบข้อมูลการด์	Card API
FR04	จัดการบัญชีเกมและบันทึกการด์ที่เปิดของได้	Vault / Account API
FR05	สั่งจอง หักสต็อก คัดส่วนลด และสร้าง PENDING	Order / Strategy
FR06	บันทึกการรับเงิน เริ่มส่งมอบ และยกเลิกตามกฎ	Order / State
FR07	แนะนำและเลือกบัญชี READY ที่ถือการด์สภาพตรงกัน	Trade Matching
FR08	บันทึก TRADE_SENT และ COMPLETED รายการรายการ	Order Item
FR09	แสดงคำสั่งซื้อเฉพาะของลูกค้าที่เข้าสู่ระบบ	Order Access Policy
FR10	เตือนใน log เมื่อสต็อกรวมของการด์เหลือไม่เกิน 2 ใบ	Observer

ความต้องการด้านคุณภาพ

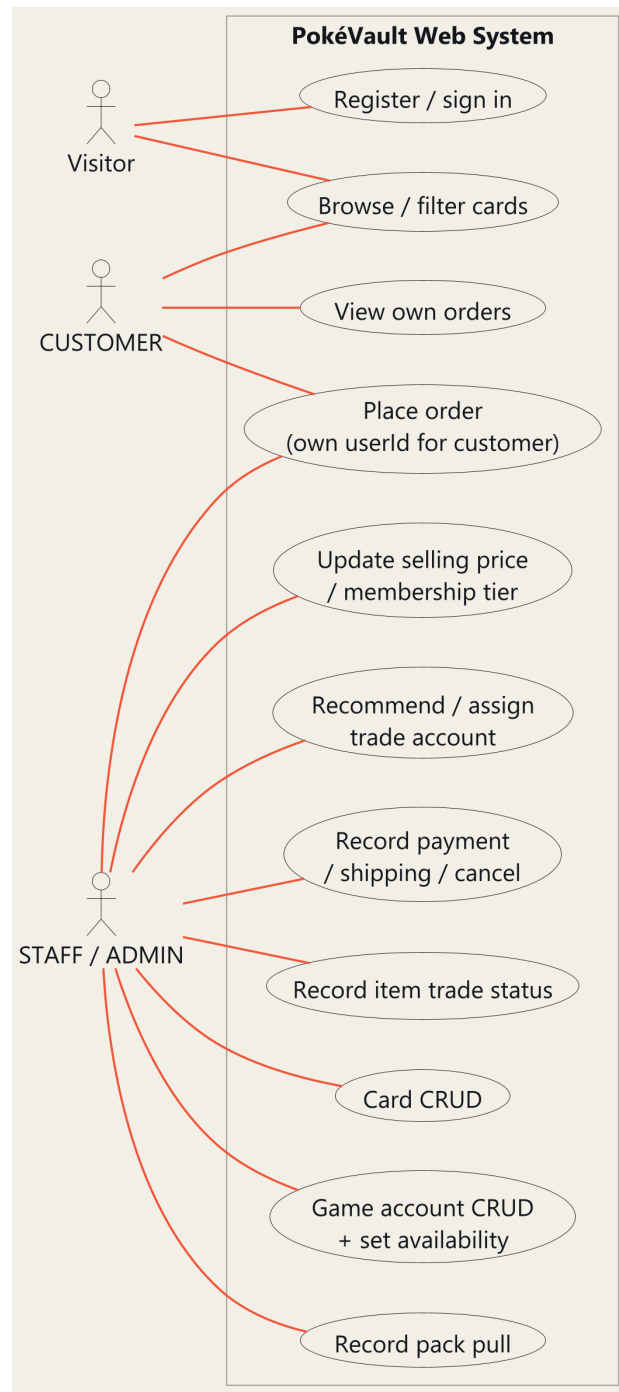
โค้ดต้องแยกหน้าที่ตาม Layer เปลี่ยนกฎธุรกิจได้โดยจำกัดจุดแก้ไข ตรวจสอบข้อมูลตั้งแต่ Request DTO และรักษาข้อมูลอ้างอิงในฐานข้อมูล หน้าเว็บต้องแสดงสถานะและขั้นตอนต่อไปให้ชัดเจน รองรับหน้าจอขนาดเล็ก และ REST API ต้องตอบผลสำเร็จและข้อผิดพลาดในรูปแบบสม่ำเสมอ

ขอบเขตภายนอกเว็บ

การรับโอนเงิน การคืนเงิน การเพิ่มเพื่อน และการส่งการด์จริงทำโดยร้านกับลูกค้าในแชทหรือเกม เว็บบันทึกผลของงานเหล่านี้ ไม่มี Payment Gateway หรือ API เชื่อมเกมโดยตรง ไม่มีตัวนับโควตาเทรดรายวัน และไม่มี Scheduler ปลด COOLDOWN อัตโนมัติ

1.5 Use Case Diagram

Actors อยู่ภายนอกขอบเขตระบบ ส่วน Use Cases อยู่ภายในกรอบ PokéVault Commerce การคำนวณส่วนลดและ Spring Events เป็นการทำงานภายใน จึงไม่แสดงเป็น Actor แยกต่างหาก STAFF และ ADMIN ใช้งานที่แสดงร่วมกันตามสิทธิ์ใน SecurityConfig



ภาพที่ 1 Use Case Diagram ของระบบ

เส้น Association แสดงการมีส่วนร่วมใน Use Case การโอนเงินและการเทรดในเกมเป็นบริบทภายนอกที่พนักงานนำผลมา
บันทึกในเว็บ ไม่ได้สื่อว่าเว็บทำรายการเหล่านั้นแทนผู้ใช้

1.7 รายละเอียด Use Case สำคัญ

UC01 สั่งจองการ์ด

ผู้ดำเนินการคือ CUSTOMER หรือพนักงานที่จองแทนลูกค้า เงื่อนไขเริ่มต้นคือมีบัญชีลูกค้าและรายการ Inventory ที่มีจำนวนพอ ผู้ใช้เลือกการ์ด จำนวน และ Friend ID ระบบตรวจเจ้าของบัญชี อ่านระดับสมาชิกจาก Profile ตรวจสอบและหักสต็อกทุก Item เก็บราคา ณ เวลาจอง คิดส่วนลด บันทึกคำสั่งซื้อ PENDING และคืนสรุปการจอง

หากรายการใดสต็อกไม่พอ Service โยน InsufficientStockException และ Transaction ย้อนกลับ ไม่ควรเหลือการหักสต็อกเฉพาะบางรายการ หากลูกค้าระบุ userId ของผู้อื่น ระบบปฏิเสธด้วย 403 ผลสำเร็จคือมีคำสั่งซื้อพร้อมยอดสุทธิ และมี OrderPlacedEvent ให้ Observer ตรวจสอบสต็อก

UC02 บันทึกการเปิดช่อง

STAFF หรือ ADMIN เลือกบัญชีเกม การ์ด สภาพ จำนวน และราคา ระบบโหลดบัญชีและการ์ด ค้น Inventory ที่ตรงทั้ง cardId accountId และ condition หากพบจะเพิ่มจำนวน หากไม่พบจะสร้างรายการใหม่และตอบ 201 กระบวนการนี้บันทึกผลเปิดช่องที่เกิดขึ้นในเกมแล้ว ไม่ได้สุ่มหรือเปิดช่องให้ในเกม

UC03 ส่งมอบการ์ดและปิดคำสั่งซื้อ

พนักงานบันทึกรับเงินเพื่อเปลี่ยน PENDING เป็น PAID จากนั้นเลือกบัญชีเกม READY ที่ถือการ์ดสภาพเดียวกันและมี Effective Stock เพียงพอ การ Assign ทำให้ Item เป็น FRIEND_PENDING เมื่อเริ่มส่งมอบ Order เป็น SHIPPING ร้านดำเนินการในเกมแล้วบันทึก TRADE_SENT และ COMPLETED ตามลำดับ Item สุดท้ายที่สำเร็จทำให้ Order เป็น COMPLETED อัตโนมัติ

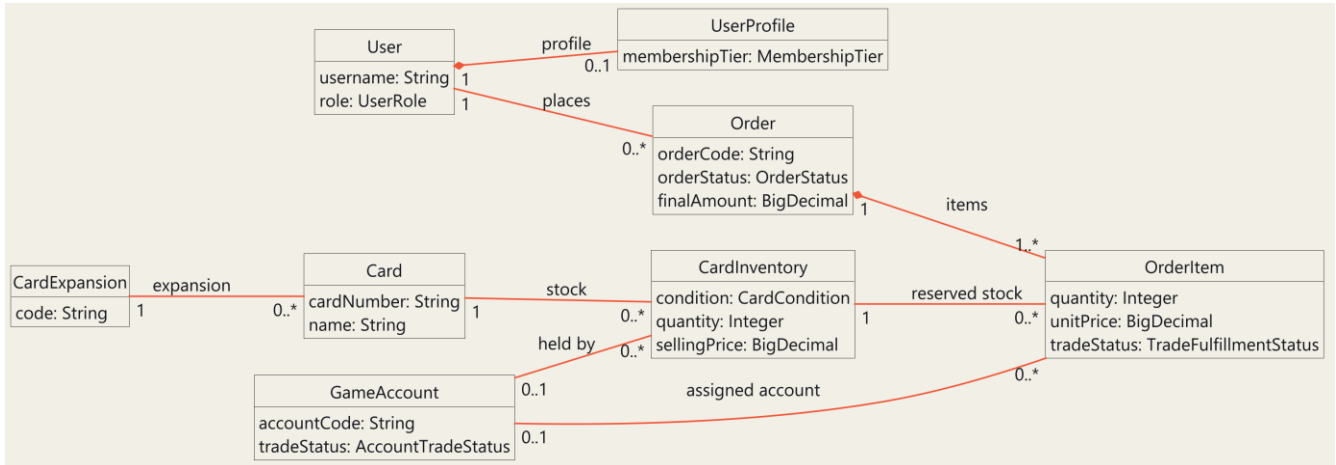
UC04 ยกเลิกคำสั่งซื้อ

พนักงานยกเลิกได้จาก PENDING หรือ PAID ระบบเปลี่ยนเป็น CANCELLED และคืนสต็อกของแต่ละ Item ภายใน Transaction เดียวกัน หากเป็น SHIPPING หรือสถานะสิ้นสุดแล้ว State ปฏิเสธด้วย 409 การคืนเงินกรณีชำระแล้วเป็นงานของร้านนอกเว็บ

แต่ละ Use Case ผ่าน Controller ไปยัง Service และ Repository ข้อผิดพลาดถูกแปลงเป็น HTTP Response โดย GlobalExceptionHandler การกำหนดสิทธิ์ที่ URL และการตรวจเจ้าของข้อมูลเป็นกลไกชั้นของการป้องกันและต้องมีทั้งสองส่วน

บทที่ 2 โครงสร้างระบบและ SOLID

Domain Model แสดงแนวคิดธุรกิจ 8 กลุ่ม ได้แก่ User UserProfile CardExpansion Card CardInventory Order OrderItem และ GameAccount การแยก Card ออกจาก Inventory ทำให้ข้อมูลการ์ดหนึ่งใบใช้ร่วมกับสต็อกหลายบัญชีและหลายสภาพได้

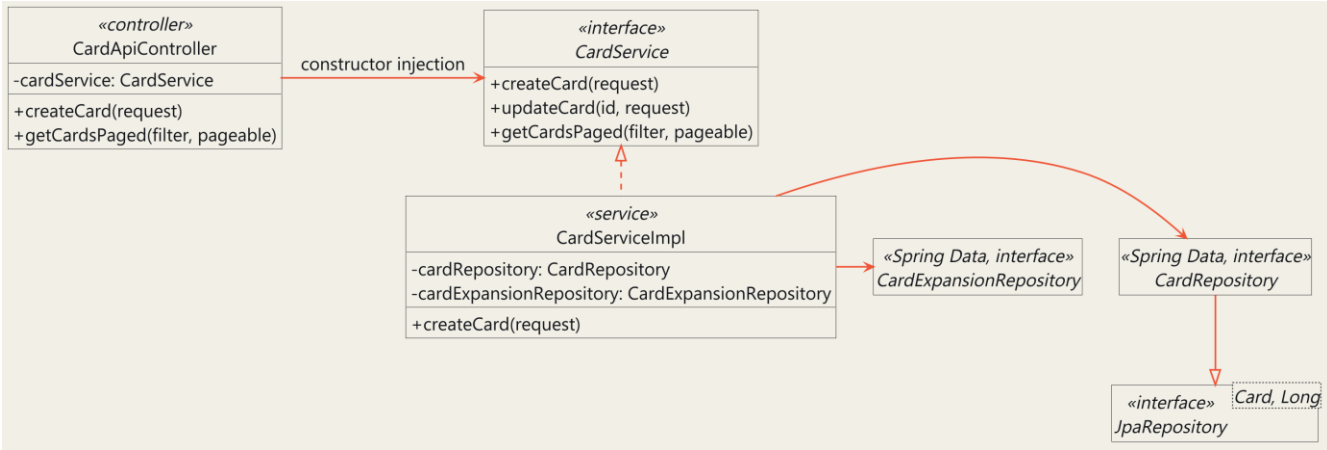


ภาพที่ 2 Domain Model และความสัมพันธ์ของข้อมูล

User มี Profile ได้ 0 หรือ 1 และมี Orders ได้หลายรายการ Order ที่สร้างผ่าน API ต้องมีอย่างน้อย 1 Item แต่ SQL ไม่บังคับจำนวนลูกชั้นต่ำ จึงเห็น 1..* ใน Domain และ 0..* ใน ER ได้โดยไม่ขัดกัน Account ของ Inventory และบัญชีที่ Assign ให้ Item เป็น Optional 0..1 Composition แสดงความเป็นเจ้าของ Profile และ Items ตาม Cascade และ orphanRemoval

2.1 Class Diagram ของ Catalog

Class Diagram แสดงโครงสร้างฝั่ง Catalog และการใช้ Interface โดย CardApiController เรียก CardService ส่วน CardServiceImpl เรียก Repository และแปลงข้อมูลระหว่าง Entity กับ DTO เส้นประหัวสามเหลี่ยมโป่งใช้แสดง Realization ของ Interface

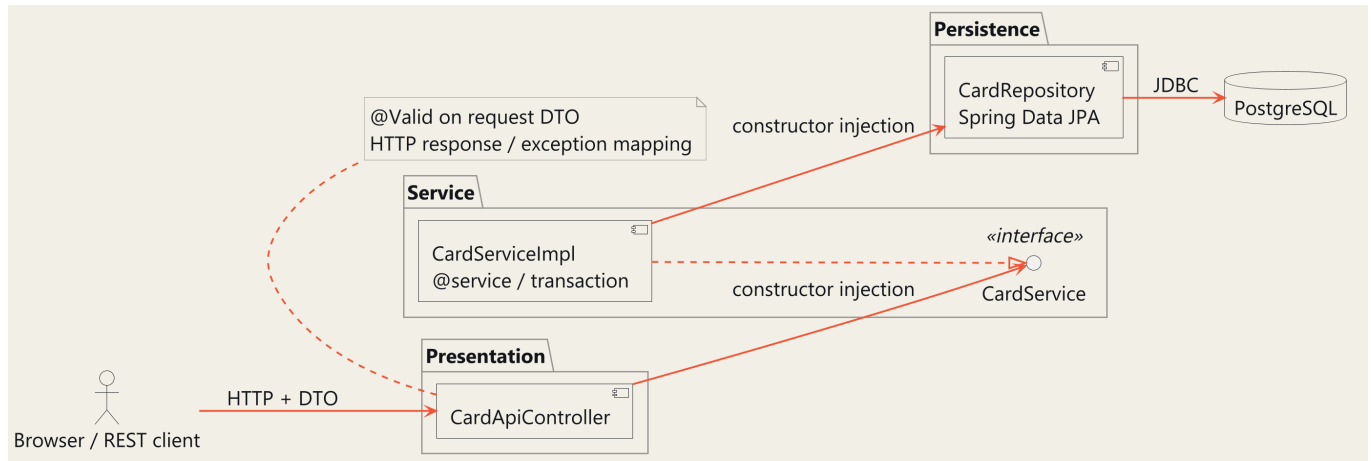


ภาพที่ 3 Class Diagram ของ Card Catalog

Request และ Response เป็นสัญญา HTTP แยกจาก Entity ข้อมูล Expansion ถูกตรวจว่ามีอยู่จริงก่อนสร้าง Card การสืบทอด JpaRepository ทำให้ Repository เป็น Interface สำหรับ Persistence ไม่ได้หมายความว่า Repository เป็นฐานข้อมูลจำลองในหน่วยความจำ

2.2 Layered Architecture

Presentation รับ HTTP และเตรียมหน้าเว็บ Service จัดการกฎธุรกิจและ Transaction Repository เข้าถึงข้อมูลผ่าน JPA และ Domain เก็บ Entity Enum และพฤติกรรมของข้อมูล DTO และ Mapper ใช้ประกอบการรับส่งข้อมูล โดยไม่เป็นทางลัดให้ Controller เข้าถึง Repository



ภาพที่ 4 Layered Architecture และเส้นทาง Request

ตารางที่ 3 หน้าที่ของแต่ละ Layer

Layer	ตัวอย่าง	หน้าที่
Presentation	CardApiController / WebViewController	รับ Request ตรวจสอบ @Valid ตอบ HTTP หรือ View
Service	CardServiceImpl / OrderServiceImpl	ตรวจสอบเงื่อนไข คำนวณราคา คุณสมบัติ และ Transaction
Repository	CardRepository / OrderRepository	CRUD Query และเชื่อม JPA กับฐานข้อมูล
Domain และ DTO	Card / Order / CardRequest / OrderResponse	โครงสร้างข้อมูลและสัญญาณที่ใช้ระหว่างขอบเขต

2.3 จาก Request ไปยัง Persistence

ตัวอย่างการสร้าง Card เริ่มที่ POST /api/v1/cards Controller รับ CardRequest ที่ผ่าน Bean Validation เรียก Service และกำหนด 201 CREATED จากนั้น Service ตรวจสอบ Expansion สร้าง Entity บันทึกผ่าน Repository และคืน CardResponse การควบคุม Transaction อยู่ที่ Service

```

83 @PostMapping
84 @Operation(summary = "Create a new card", description = "Add a new Pokemon card to the catalog")
85 public ResponseEntity<ApiResponse<CardResponse>> createCard(@Valid @RequestBody CardRequest request) {
86     CardResponse created = cardService.createCard(request);
87     return ResponseEntity.status(HttpStatus.CREATED)
88         .body(ApiResponse.ok("Card created successfully", created));
89 }

```

```

169
170 @Override
171 @Transactional
172 public CardResponse createCard(CardRequest request) {
173     CardExpansion expansion = cardExpansionRepository.findById(request.getExpansionId())
174         .orElseThrow(() -> new ResourceNotFoundException("CardExpansion", "id", request.getExpansionId()));
175
176     ...
177
187
188     Card savedCard = cardRepository.save(card);
189     return CardResponse.fromEntity(savedCard);
190 }
191

```

เหตุผลที่เลือกโครงสร้างนี้

Controller ไม่ต้องรู้วิธี Query หรือจัดเก็บ Entity การเปลี่ยนรูปแบบหน้าเว็บจึงไม่บังคับให้เปลี่ยนกฎสร้างการ์ด และ Service ทดสอบได้โดย Mock Repository ใน Unit Test ส่วนการทดสอบการ Mapping กับ SQL ทำแยกใน Persistence Test

ขอบเขตของ Interface

CardService กับ CardServiceImpl เป็น Interface และ Implementation ภายใน Service Layer เดียวกัน เส้นทางในแผนภาพสื่อการพึ่งพา abstraction ไม่ได้หมายความว่า Layer เพิ่มอีกชั้นระหว่าง Interface กับ Implementation การใช้ @RequiredArgsConstructor สร้าง Constructor สำหรับ final dependencies ทำให้ Dependency ของคลาสเห็นได้ชัด

2.4 SOLID หลักการ S และ O

Single Responsibility Principle

WebViewController รับ Request และส่ง Model ให้ View ขณะที่ WebPageServiceImpl รวบรวมข้อมูลเป็น View DTO และ LowStockObserver ติดตามสต็อกจาก Event แยกจากการสร้างคำสั่งซื้อ ดังนั้นการปรับข้อมูลที่แสดงบนเว็บและการปรับเกณฑ์สต็อกต่ำเกิดคนละจุดกับ HTTP Routing และ Order Creation

ตารางที่ 4 หลักฐาน Single Responsibility

คลาส	ความรับผิดชอบ	ตำแหน่งหลักฐาน
WebViewController	Routing และ Model ของหน้าเว็บ	modules/web/controller/WebViewController.java:28
WebPageServiceImpl	ข้อมูลและ View Models ของแต่ละหน้า	modules/web/service/WebPageServiceImpl.java:44
LowStockObserver	ตรวจเกณฑ์สต็อกหลัง OrderPlacedEvent	modules/vault/observer/LowStockObserver.java:39

Open Closed Principle

DiscountService พึ่ง DiscountStrategy และค้น Strategy ที่ supports ระดับสมาชิก การเพิ่มสูตรส่วนลดจึงทำได้ผ่าน Implementation ใหม่ โดยไม่รวมเงื่อนไขของทุกสูตรใน OrderService ได้ด้วยการเลือก Strategy ทำหน้าที่ประสานงาน ส่วนแต่ละ Strategy รับผิดชอบสูตรของตน

```

28 public DiscountStrategy getApplicableStrategy(MembershipTier tier) {
29     return strategies.stream()
30         .filter(strategy -> strategy.supports(tier))
31         .findFirst()
32         .orElseGet(() -> strategies.stream()
33             .filter(strategy -> strategy.supports(MembershipTier.REGULAR))
34             .findFirst()
35             .orElseThrow(() -> new IllegalStateException("No suitable discount strategy found for tier: " + tier)));
36 }

```

ขอบเขตของการขยายยังมีอยู่ หากเพิ่มระดับสมาชิกใหม่ต้องปรับ MembershipTier และสัญญา UI หรือข้อมูลที่เกี่ยวข้อง การใช้ OCP จึงช่วยจำกัดจุดเปลี่ยนกฎ ไม่ได้ทำให้ทุกส่วนของระบบขยายได้โดยไม่มีการแก้ไขใดๆเลย

2.5 SOLID หลักการ L และ I

Liskov Substitution Principle

RegularDiscountStrategy VipDiscountStrategy และ WholesaleDiscountStrategy ใช้ Contract เดียวกัน ผู้เรียกส่ง subtotal แบบ BigDecimal และได้รับจำนวนส่วนลดแบบ BigDecimal Strategies ตรวจสอบ subtotal ที่เป็น null หรือไม่มากกว่าศูนย์ตามพฤติกรรมที่กำหนด และบดเศษสองตำแหน่ง ผู้เรียกไม่ต้องเปลี่ยนชนิดข้อมูลเมื่อเปลี่ยนสูตร

ตารางที่ 5 ตัวอย่าง Contract ของ DiscountStrategy

Implementation	Tier ที่รองรับ	ส่วนลดของยอด 1000 บาท
RegularDiscountStrategy	REGULAR	0.00 บาท
VipDiscountStrategy	VIP	100.00 บาท
WholesaleDiscountStrategy	WHOLESALE	150.00 บาท

ใน State Pattern การปฏิเสธ Action เป็นส่วนหนึ่งของ Contract เช่น ShippingOrderState ไม่อนุญาต cancel และสถานะสิ้นสุดไม่อนุญาตเลื่อนต่อ จึงต้องอธิบายเงื่อนไขของ Contract ไม่อ้างว่าทุก State ทำ Action ทุกชนิดได้ การปฏิเสธใช้ Exception ของโดเมน ไม่ใช่ UnsupportedOperationException

Interface Segregation Principle

Interface แยกตามงานของ Catalog Vault Order Trade และ Web เช่น CardService ใช้กับข้อมูลการ์ด GameAccountService ใช้กับบัญชีเกม และ WebPageService ใช้เตรียมข้อมูลหน้าเว็บ ผู้เรียกแต่ละกลุ่มไม่ต้องพึ่ง Interface รวมทุกความสามารถของระบบ

ตารางที่ 6 การแบ่ง Service Interfaces

Interface	ผู้ใช้หลัก	ขอบเขต
CardService	CardApiController	ค้นหา กรอง และ CRUD ของ Catalog
GameAccountService	GameAccountApiController	CRUD บัญชีและบันทึกเปิดของ
OrderService	OrderApiController	จองและอัปเดตคำสั่งซื้อ
TradeMatchingService	TradeMatchingApiController	Recommendation และ Assign บัญชี
WebPageService	WebViewController	รวบรวม View DTO

2.6 SOLID หลักการ D และ Constructor Injection

Controller พึ่ง Service Interface และ Service พึ่ง Repository Interface จึงสามารถแทน Dependency ด้วย Mock ในการทดสอบได้ Spring รับผิดชอบประกอบ Object ผ่าน Constructor ไม่ใช้การฉีด Dependency ลง Private Field ด้วย @Autowired

```

23 @RestController
24 @RequestMapping("/api/v1/cards")
25 @RequiredArgsConstructor
26 @Tag(name = "Card Catalog API", description = "Endpoints for browsing, filtering, and searching Pokemon TCG Pocket cards and
expansions")
27 public class CardApiController {
28
29     private final CardService cardService;
30

```

```

28 @Service
29 @RequiredArgsConstructor
30 @Transactional(readOnly = true)
31 public class CardServiceImpl implements CardService {
32
33     private final CardRepository cardRepository;
34     private final CardExpansionRepository cardExpansionRepository;
35
36     @Override
37     public List<CardResponse> getAllCards(CardFilterRequest filter) {
38         List<Card> cards = cardRepository.findAll();

```

การทำงานร่วมกับ Spring

@RequiredArgsConstructor สร้าง Constructor ที่รับ final fields ทั้งหมด ในการสร้าง Bean Spring เลือก Implementation ที่เหมาะสมแล้วส่งให้ Constructor การพึ่ง Interface ทำให้ Controller ไม่สร้าง CardServiceImpl ด้วย new และไม่ผูกกับวิธีจัดเก็บข้อมูลโดยตรง

ข้อจำกัดของ Implementation

SecurityConfig รับ CustomDetailsService ซึ่งเป็น Concrete Class แม้คลาสนี้ Implement UserDetailsService ของ Spring Security อยู่แล้ว ส่วนนี้เป็นจุดที่ยังปรับให้พึ่ง Interface ได้ รายงานจึงแสดงการประยุกต์ SOLID พร้อมขอบเขตที่พบจากโค้ด แทนการรับรองว่าไม่มีการละเมิดในทุกคลาส

เอกสาร doc/solid-analysis.md ระบุหลักการ ไฟล์ บรรทัด และเหตุผล ใช้ร่วมกับ Class Diagram และตัวอย่างโค้ดใน
บทนี้เพื่อตรวจสอบการออกแบบได้

บทที่ 3 Design Patterns และเหตุการณ์เลือกใช้

การเลือก Pattern อิงปัญหาของระบบ โดย Enterprise Patterns จัดโครงสร้างงานและขอบเขตข้อมูล ส่วน GoF Patterns ควบคุมพฤติกรรมที่เปลี่ยนตามระดับสมาชิก สถานะ และเหตุการณ์ GoF ทั้งสามแบบอยู่ในกลุ่ม Behavioral ตามข้อกำหนดของรายวิชา

Enterprise Patterns รายการที่ 1 ถึง 3

Layered Architecture

ใช้แยกรับส่ง HTTP ธุรกรรม และ Persistence ลดผลกระทบเมื่อเปลี่ยนหน้าเว็บหรือวิธี Query ตัวอย่าง คือ CardApiController → CardService → CardRepository และ WebViewController → WebPageService โครงสร้างอยู่ในภาพ Layered ของบทที่ 2

Model View Controller

WebViewController เป็น Controller เตรียม Model จาก WebPageService แล้วเลือก Thymeleaf Template เช่น cards.html และ orders.html View รับผิดชอบการแสดงผล ขณะที่ JavaScript ใช้ REST API เพื่อทำ Action และปรับหน้าจอ จึงไม่วางกฎส่วนลดหรือกฎ State ไว้เป็นแหล่งตัดสินใจหลักใน View

Repository

CardRepository GameAccountRepository และ OrderRepository สืบทอด JpaRepository ทำให้ Service ใช้สัญญา CRUD และ Custom Query แทนการเขียน JDBC ทุกจุด ใช้ JPQL รวมสต็อกของการดักข้มบัญชี Repository สามารถทำงานกับ H2 หรือ PostgreSQL ตาม DataSource ที่ตั้งค่า

ตารางที่ 7 Enterprise Patterns กลุ่มโครงสร้าง

Pattern	ปัญหาที่แก้	หลักฐาน
Layered	การผสม HTTP กับธุรกรรมและฐานข้อมูล	Controller / Service / Repository
MVC	การผสมข้อมูลหน้าเว็บกับการแสดงผล	WebViewController / WebPageService / Templates
Repository	การกระจาย SQL และ Data Access ไปหลายจุด	repository/*Repository.java

Entity และ DTO เป็นคนละบทบาท Entity มี Mapping และความสัมพันธ์ของฐานข้อมูล ส่วน DTO จำกัดข้อมูลที่ส่งผ่านขอบเขต Controller ไม่ควรใช้ Entity เป็น Request Contract โดยตรง

3.1 Enterprise Patterns รายการที่ 4 ถึง 6

Service Layer

OrderServiceImpl ทำ Use Case สั่งจองครบทั้งการอ่านผู้ใช้ หักสต็อก เก็บราคา คิดส่วนลด บันทึก Order และเผยแพร่ Event ภายใน Transaction เมื่อบางขั้นล้มเหลว การเปลี่ยนแปลงฐานข้อมูลต้องย้อนกลับพร้อมกัน Service จึงเป็นขอบเขตการทำงานที่ทั้ง REST และหน้าเว็บพึงพา

DTO และ Mapper

CardRequest กำหนดข้อมูลและ Validation ที่รับจาก HTTP ขณะที่ CardResponse และ OrderResponse กำหนดผลตอบกลับ เมธอด fromEntity และ WebViewModelapper แปลง Entity เป็นข้อมูลสำหรับ API และ View การแยกนี้ช่วยควบคุมข้อมูล เช่นไม่ส่ง password ของ User หรือกราฟความสัมพันธ์ทั้งหมดไปยัง Browser

Dependency Injection

Spring ประกอบ Controller และ Service ผ่าน Constructor Injection รวมถึง inject List<DiscountStrategy> ให้ DiscountService ช่วยให้เลือก Strategy ตามข้อมูล และทำ Unit Test โดยส่ง Mock หรือ Implementation ที่เตรียมไว้ การใช้ DI ทำให้ Dependency อยู่ใน Constructor และลดการสร้าง Object ที่ผูกกับ Concrete Class ภายในผู้เรียก

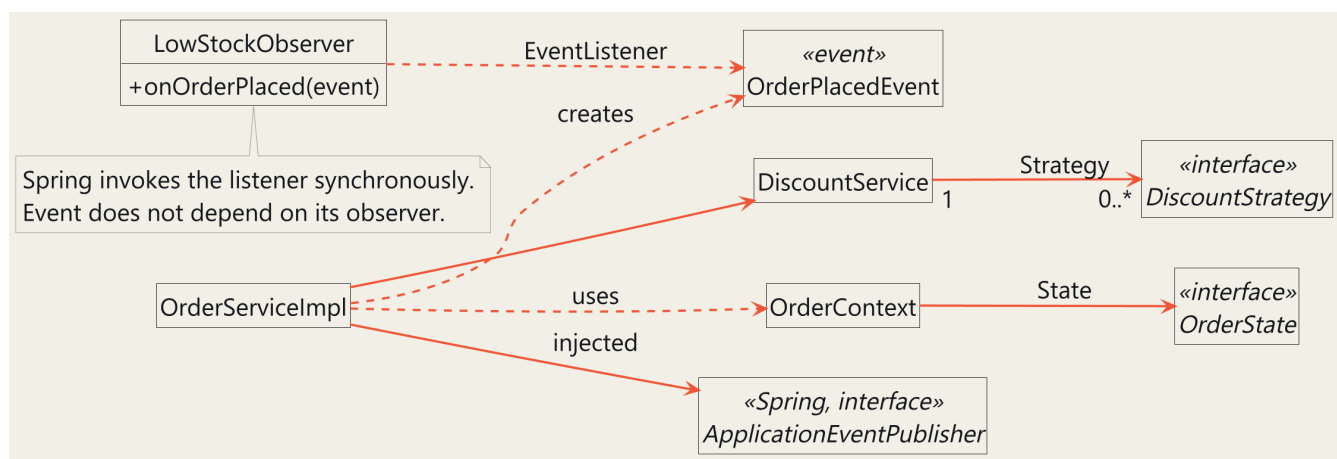
ตารางที่ 8 Enterprise Patterns กลุ่มบริการและข้อมูล

Pattern	จุดใช้งาน	เหตุผล
Service Layer	OrderServiceImpl / GameAccountServiceImpl	รวม Business Rules และ Transaction ต่อ Use Case
DTO และ Mapper	CardRequest / OrderResponse / WebViewModelapper	แยกสัญญา HTTP และ View จาก Entity
Dependency Injection	Constructor / @RequiredArgsConstructor	ประกอบ Dependencies และรองรับ Mock Test

ต้นทุนของการเลือกใช้

การแยก Interface DTO และ Mapper ทำให้มีไฟล์มากขึ้น แต่ช่วยให้เห็นความรับผิดชอบและเขียน Test แยกชั้นได้ กลุ่มเลือกใช้ตามขนาดและโดเมนของงาน โดยยังอยู่ใน Application เดียว ไม่ได้แยกเป็น Microservices ซึ่งจะเพิ่มความซับซ้อนด้านการสื่อสารและการจัดการ Transaction

3.2 Class Diagram และตำแหน่ง GoF Patterns



ภาพที่ 5 Class Diagram แสดง Strategy State และ Observer

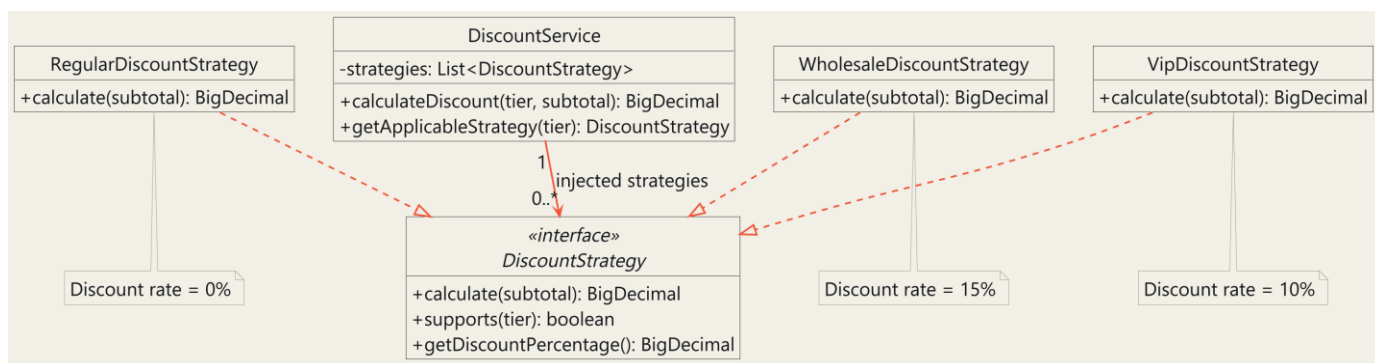
ตารางที่ 9 ตำแหน่งและบทบาทของ GoF Patterns

Pattern	ผู้เรียกหรือ Context	Abstraction และ Implementation	เหตุผลเลือกใช้
Strategy	OrderServiceImpl / DiscountService	DiscountStrategy และ 3 Tier Strategies	เปลี่ยนสูตรส่วนลดโดยจำกัดการแก้ Order Service
State	OrderServiceImpl / OrderContext	OrderState และ 5 Order States	ควบคุม Action ที่อนุญาตและสถานะสิ้นสุด
Observer	ApplicationEventPublisher	OrderPlacedEvent / LowStockObserver	แยกการติดตามสต็อกจากการสร้างคำสั่งซื้อ

เส้น Dependency ชี้จากคลาสที่ใช้ไปยังชนิดข้อมูลหรือ Interface ที่พึ่งพา LowStockObserver พึ่ง OrderPlacedEvent ในฐานะข้อมูล Event ส่วน Service พึ่ง ApplicationEventPublisher โดยไม่ต้องเรียก Observer แต่ละตัวโดยตรง

3.3 Strategy Pattern

โจทย์คือส่วนลดต่างกันตาม MembershipTier หากรวมทุกสูตรไว้ใน OrderService การเพิ่มหรือปรับสูตรจะกระทบกระบวนการจอง กลุ่มจึงแยก Contract ของการคำนวณส่วนลด และให้แต่ละ Strategy รับผิดชอบการรองรับ Tier กับสูตรของตน



ภาพที่ 6 Class Diagram ของ Discount Strategy

DiscountService เลือก Strategy จาก List ที่ Spring inject โดยค้น **supports(tier)** แล้วเรียก **calculate(subtotal)** ผู้สร้างคำสั่งซื้อจึงเห็นเพียงจำนวนส่วนลด ไม่จำเป็นต้องรู้ชื่อ Concrete Strategy ระดับสมาชิกอ่านจาก **UserProfile** ที่ Server ไม่รับอัตราส่วนลดจากลูกค้า

REGULAR ลด 0 เปอร์เซ็นต์ VIP ลด 10 เปอร์เซ็นต์ และ WHOLESALE ลด 15 เปอร์เซ็นต์ หากไม่พบ Tier ที่ตรง ระบบพยายามใช้ REGULAR และหากไม่มี Strategy ที่รองรับเลยจะโยน **IllegalStateException** การเพิ่ม Tier ใหม่ยังต้องรักษาสัญญาข้อมูลและการจัดการสมาชิก

3.4 ตัวอย่างโค้ดและการคิดส่วนลด

```

14 private final List<DiscountStrategy> strategies;
15
16 public DiscountService(List<DiscountStrategy> strategies) {
17     this.strategies = strategies != null ? strategies : List.of();
18 }
19
20 public BigDecimal calculateDiscount(MembershipTier tier, BigDecimal subtotal) {
21     if (subtotal == null || subtotal.compareTo(BigDecimal.ZERO) <= 0) {
22         return BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP);
23     }
24     DiscountStrategy strategy = getApplicableStrategy(tier);
25     return strategy.calculate(subtotal);
26 }

```

```

12 private static final BigDecimal DISCOUNT_RATE = new BigDecimal("0.15");
13 private static final BigDecimal DISCOUNT_PERCENTAGE = new BigDecimal("15.00");
14
15 @Override
16 public BigDecimal calculate(BigDecimal subtotal) {
17     if (subtotal == null || subtotal.compareTo(BigDecimal.ZERO) <= 0) {
18         return BigDecimal.ZERO.setScale(2, RoundingMode.HALF_UP);
19     }
20     return subtotal.multiply(DISCOUNT_RATE).setScale(2, RoundingMode.HALF_UP);
21 }

```

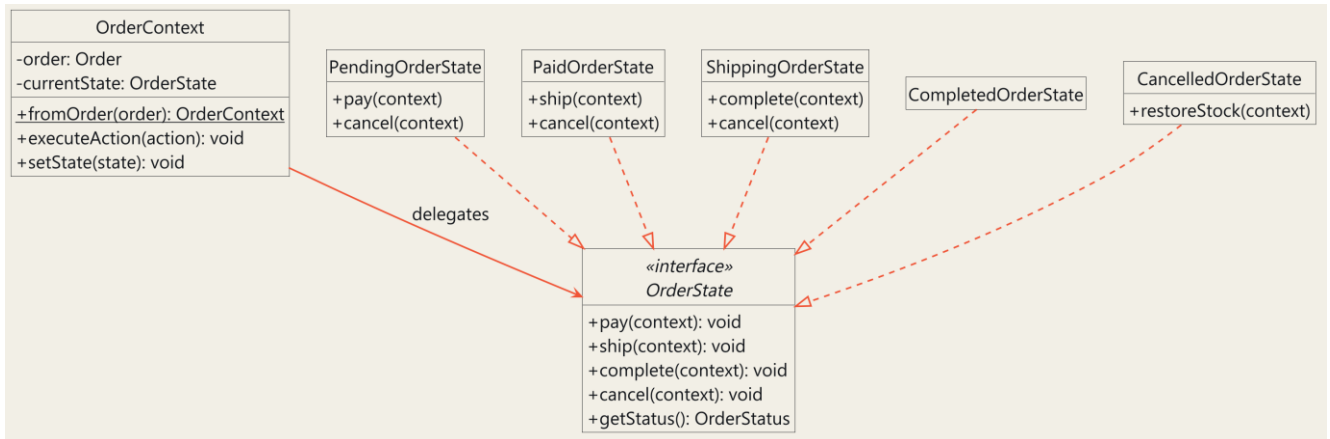
ตารางที่ 10 ตัวอย่างการคำนวณราคาเป็นบาท

Tier	ยอดรวม	ส่วนลด	ยอดสุทธิ
REGULAR	1000.00	0.00	1000.00
VIP	1000.00	100.00	900.00

ตัวเลขในตารางเป็นตัวอย่างอธิบายสูตร ส่วนลดคำนวณด้วย BigDecimal และ setScale(2, HALF_UP) OrderService รวม subtotal ของทุกรายการ แล้วหัก discountAmount โดยจำกัดยอดสุทธิไม่ให้ต่ำกว่าศูนย์ ราคาของ OrderItem ถูกเก็บ ณ เวลาจอง จึงไม่เปลี่ยนแปลงตามราคาล่าสุดที่พนักงานแคชเชียร์

3.5 State Pattern

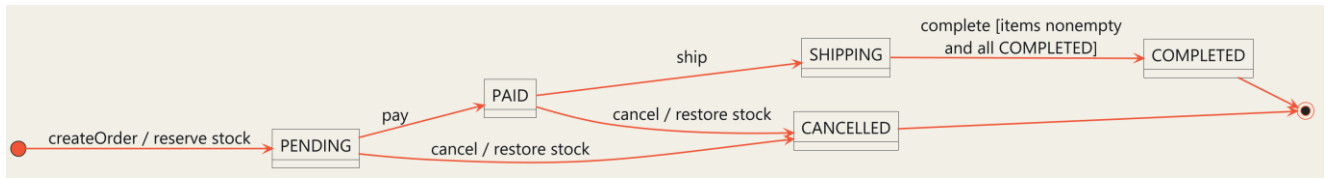
OrderContext ถือ Order และ currentState ซึ่งเป็น OrderState ส่วน Concrete States กำหนด Action ที่อนุญาต ในสถานะนั้น การสร้าง Context จาก Entity อ่าน OrderStatus แล้วเลือก State ที่ตรง จึงยังใช้ Enum สำหรับบันทึกสถานะ แต่ พฤติกรรมไม่ได้มีเพียงการตั้งค่า Enum



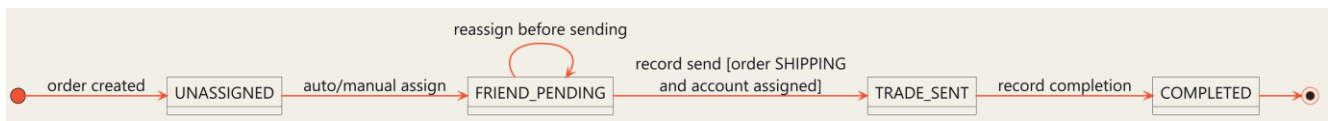
ภาพที่ 7 Class Diagram ของ Order State Pattern

Pending รองรับ pay และ cancel Paid รองรับ ship และ cancel Shipping รองรับ complete เมื่อทุกรายการเสร็จ Completed และ Cancelled เป็นสถานะสิ้นสุด Default Action ที่ไม่อนุญาตโยน `InvalidOrderStateException` ซึ่งแปลงเป็น HTTP 409 ในชั้น Exception Handler

3.6 สถานะ Order และ OrderItem



ภาพที่ 8 State Diagram ของคำสั่งซื้อ



ภาพที่ 9 State Diagram ของการส่งมอบรายการ

Order แสดงความคืบหน้าของคำสั่งซื้อทั้งฉบับ ส่วน OrderItem แสดงการส่งมอบการ์ดแต่ละรายการ การ Assign ทำให้ Item เป็น **FRIEND_PENDING** ก่อนบันทึก **TRADE_SENT** ต้องมีบัญชีที่ Assign และ Order ต้องเป็น **SHIPPING** จากนั้นจึงบันทึก **COMPLETED** ได้

เมื่อ Item สุดท้ายสำเร็จ OrderService ตรวจสอบว่ารายการไม่ว่างและทุก Item เป็น **COMPLETED** แล้วเรียก `Context.complete` ภายในงานเดียวกัน การส่งสถานะ Item เดิมเข้าคืนข้อมูลปัจจุบันโดยไม่ประมวลผลซ้ำ แต่การสั่ง `complete` Order ที่อยู่ใน **COMPLETED** แล้วผ่าน Action ของ State จะถูกปฏิเสธ

3.7 Guard และการคืนสต็อก

```

12 public void complete(OrderContext context) {
13     if (context != null && context.getOrder() != null) {
14         Order order = context.getOrder();
15         if (order.getItems() == null || order.getItems().isEmpty()) {
16             throw new TradeStateConflictException("Cannot complete order: order has no items");
17         }
18         boolean allItemsCompleted = order.getItems().stream()
19             .allMatch(item -> item.getTradeStatus() == TradeFulfillmentStatus.COMPLETED);
20         if (!allItemsCompleted) {
21             throw new TradeStateConflictException(
22                 "Cannot complete order: not all items have reached COMPLETED trade status");
23         }
24     }
25     // เมื่อการเทรดในเกมเสร็จสิ้นสมบูรณ์ -> สลับไปสถานะ COMPLETED
26     context.setState(new CompletedOrderState());
27 }

```

เงื่อนไขก่อนปิดคำสั่งซื้อ

ShippingOrderState ตรวจสอบว่ามี Items และทุก Item ส่งมอบสำเร็จ เพื่อป้องกันการปิด Order ก่อนลูกค้าได้รับการด์ครบ หากไม่ผ่านเงื่อนไขจะตอบ 409 พร้อมข้อความอธิบาย ผัง UI ควรใช้ผลตอบกลับจาก Server และแสดงสถานะใหม่เมื่อการส่งมอบ Item สุดท้ายปิด Order อัตโนมัติ

การยกเลิก

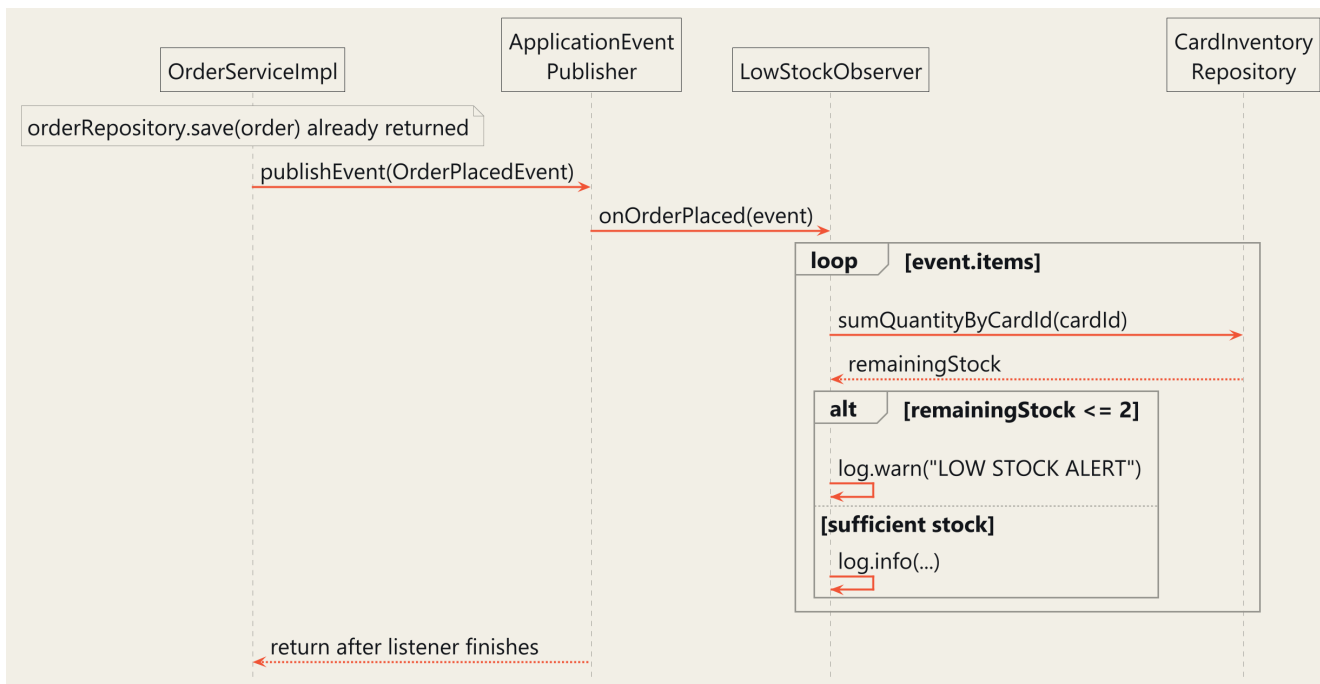
PENDING และ PAID สามารถเปลี่ยนเป็น CANCELLED ได้ โดย CancelledOrderState คืนจำนวนให้ Inventory และ OrderService บันทึกการคืนสต็อกภายใน Transaction หาก Order เป็น SHIPPING จะไม่อนุญาตให้ cancel เพราะเริ่มกระบวนการส่งมอบแล้ว การคืนเงินกรณีชำระแล้วไม่ได้ทำผ่านระบบนี้

สถานะความพร้อมของบัญชีเกม

GameAccount มี READY BUSY_TRADING COOLDOWN และ SUSPENDED ให้พนักงานตั้งค่าผ่าน API แต่ไม่มี Transition Table บังคับลำดับหรือ Timer อัตโนมัติ จึงไม่ตีความ Enum ชุดนี้เป็น State Pattern อีกชุด การ Assign ใช้เฉพาะบัญชี READY ส่วน Recommendation อาจแสดงบัญชีที่ไม่ READY พร้อมผลว่าใช้จับคู่ไม่ได้

3.8 Observer Pattern

OrderService สร้าง OrderPlacedEvent หลังบันทึกคำสั่งซื้อ และส่งผ่าน ApplicationEventPublisher LowStockObserver รับ Event ด้วย @EventListener แล้ว Query สต็อกรวมของการค้าจากทุก Inventory การตรวจสอบสต็อกจึงไม่ต้องวางใน Controller หรือให้ OrderService เรียก Observer โดยตรง



ภาพที่ 10 Sequence Diagram ของ OrderPlacedEvent และ LowStockObserver

Listener เป็น Synchronous Callback ภายใน Transaction ก่อน Commit เพราะไม่ได้ใช้ @Async หรือ @TransactionalEventListener(AFTER_COMMIT) เมื่อเหลือไม่เกิน 2 ใบจะเขียน WARN log ไม่มีการส่งอีเมลหรือ Push Notification ใน Implementation ปัจจุบัน

3.9 หลักฐานโค้ด Observer

```

30 public static final int LOW_STOCK_THRESHOLD = 2;
31
32 private final CardInventoryRepository cardInventoryRepository;
...
38 @EventListener
39 public void onOrderPlaced(OrderPlacedEvent event) {
40     if (event == null || event.getItems() == null || event.getItems().isEmpty()) {
41         return;
42     }
...
47 for (OrderItem item : event.getItems()) {
48     CardInventory inventory = item.getInventory();
49     if (inventory != null && inventory.getCard() != null) {
50         Card card = inventory.getCard();
51         int remainingStock = cardInventoryRepository.sumQuantityByCardId(card.getId());
52
53         if (remainingStock <= LOW_STOCK_THRESHOLD) {
54             log.warn("⚠️ LOW STOCK ALERT: Card '{}' ({} has only {} copies left in vault (threshold: {})",
55                     card.getName(), card.getCardNumber(), remainingStock, LOW_STOCK_THRESHOLD);
56         } else {
57             log.info("Card '{}' ({} stock is sufficient: {} copies remaining in vault.",
58                     card.getName(), card.getCardNumber(), remainingStock);
59         }
60     }
61 }

```

เหตุผลการเลือกใช้

เหตุการณ์การจองและการติดตามสต็อกเป็นคุณสมบัติที่ซับซ้อน หากในอนาคตเพิ่มผู้รับ Event สำหรับ Audit หรือแจ้งเตือนรูปแบบอื่น สามารถเพิ่ม Listener ได้ แต่ต้องเลือกขอบเขต Transaction ให้เหมาะสม โดยเฉพาะงานส่งข้อความภายนอกที่ไม่ควรเกิดก่อนทราบผล Commit

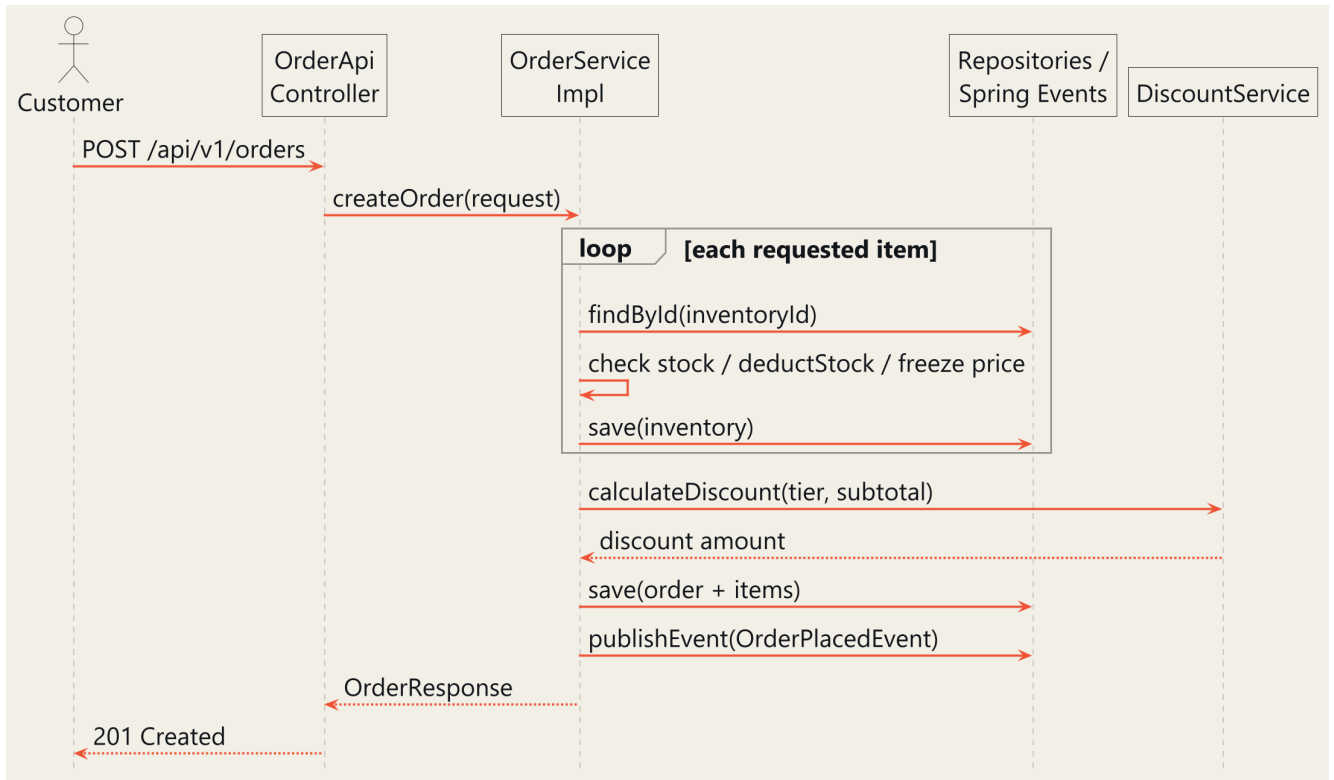
ความเชื่อมโยงกับ Repository

CardInventoryRepository.sumQuantityByCardId ใช้ SUM และ COALESCE ใน JPQL เพื่อได้สต็อกรวม แม้การ์ดนั้นอยู่หลายบัญชี Threshold จึงอิงจำนวนคงเหลือของการ์ดทั้งร้าน ส่วนข้อความสต็อกต่ำในหน้าคลังเป็นการแสดงข้อมูล UI ไม่ใช่การที่ Observer ส่งข้อความเข้า Browser โดยตรง

ข้อได้เปรียบคือแยกผู้สร้าง Event ออกจากผู้ตอบสนอง ข้อควรระวังคือ Callback ปัจจุบันทำงานใน Request เดียวกัน และอาจ Query ซ้ำเมื่อมีหลาย Item ของการ์ดเดียวกัน หากขยายปริมาณงานควรรวม cardId ที่ซ้ำและออกแบบการส่งต่อ Event หลัง Commit ให้ชัดเจน

บทที่ 4 การทำงานของ Use Cases

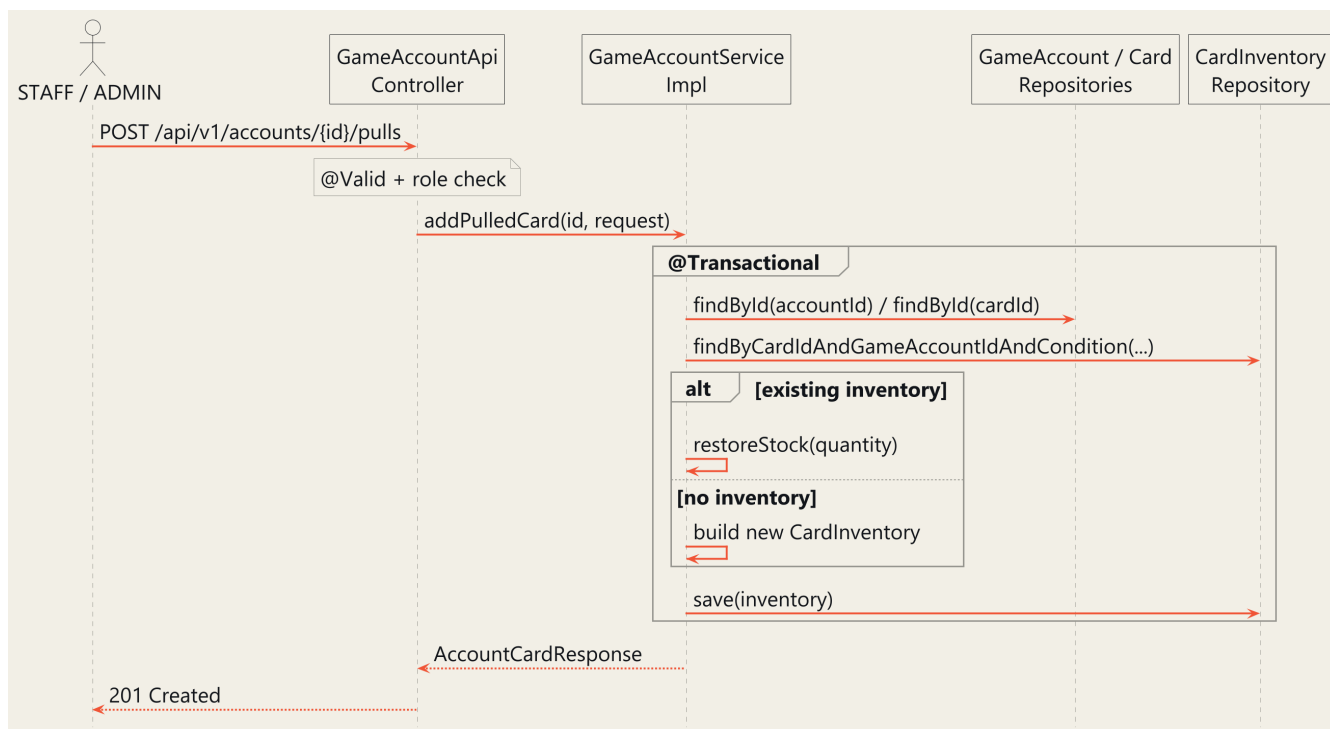
Sequence Diagram แสดงลำดับ Message ระหว่าง Object ในการทำ Use Case ส่วน Activity Diagram แสดงขั้นตอนและจุดตัดสินใจ ทั้งสองมุมมองใช้ตรวจสอบว่าลำดับงานใน Service ตรงกับกฎธุรกิจ ภาพในบทนี้มาจากไฟล์ PlantUML รายละเอียด ไม่ใช่ Flowchart แทนสัญลักษณ์ UML



ภาพที่ 11 Sequence Diagram การสั่งจองการ์ด

ลำดับจริงคืออ่านผู้ใช้และ Tier → วนตรวจและหักสต็อกแต่ละ Item พร้อมเก็บราคา → คำนวณส่วนลด → บันทึก Order → Publish Event → คำนวณ DTO การคิดส่วนลดอยู่หลังรวมราคาและก่อนบันทึกผลสุทธิ การทำรายการอยู่ใน @Transactional และ Event Listener ปัจจุบันทำงานก่อน Commit

4.1 Sequence การบันทึกการเปิดซอง

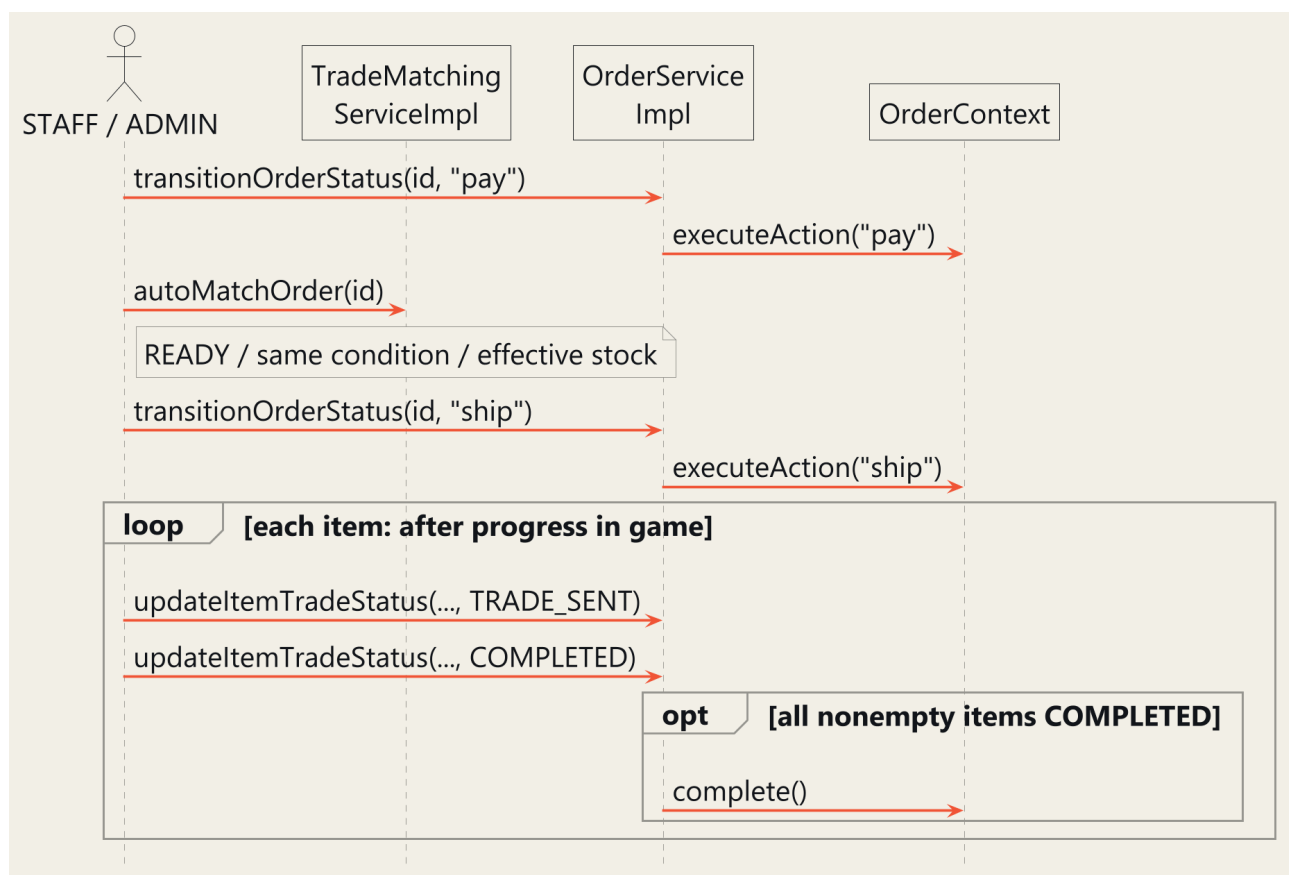


ภาพที่ 12 Sequence Diagram การบันทึกการ์ดที่เปิดซองได้

GameAccountServiceImpl.addPulledCard โหลด GameAccount และ Card จาก Repository แล้วค้น Inventory ที่ตรง cardId accountId และ condition แขนง alt แสดงกรณีเพิ่มจำนวนในรายการเดิมกับกรณีสร้างรายการใหม่ จากนั้นบันทึกและคืน AccountCardResponse พร้อม HTTP 201

กรณีไม่พบบัญชีหรือการ์ดจะตอบ 404 ข้อมูลจำนวนและราคาตรวจใน AddPulledCardRequest เช่น quantity อย่างน้อย 1 และราคาต้องไม่ติดลบ การรักษา key ของการ์ด บัญชี และสภาพช่วยไม่ให้เอาสต็อกคนละสภาพมารวมเป็นรายการเดียว

4.2 Sequence การส่งมอบและปิด Order



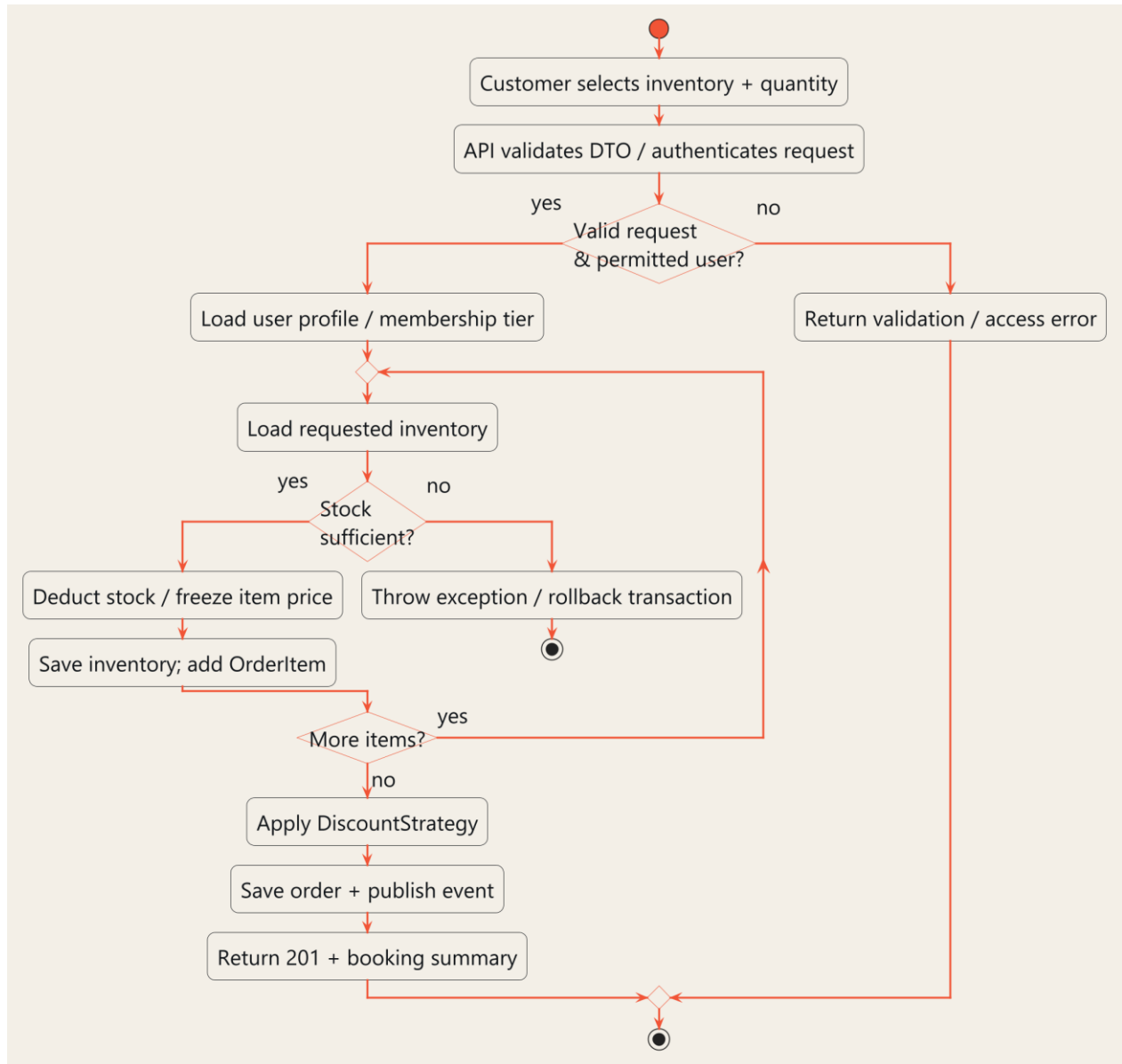
ภาพที่ 13 Sequence Diagram การรับเงิน จับคู่ และส่งมอบ

พนักงานยืนยันการรับเงิน เลือกบัญชี และเริ่มส่งมอบผ่าน Action pay และ ship ใน OrderContext ระหว่างวนแต่ละ Item ร้านทำงานในเกมแล้วบันทึก TRADE_SENT และ COMPLETED ผ่าน OrderService เมื่อทุก Item สำเร็จ ระบบเรียก complete อัตโนมัติ

TradeMatching เลือกเฉพาะบัญชี READY ที่ถือ Card และ Condition ตรง พร้อม Effective Stock เพียงพอ หากเลือก Inventory เดิมที่ถือการจองอยู่แล้ว จะนับจำนวนที่จองไว้กลับมาประกอบการตัดสินใจ หากย้ายบัญชี ระบบคืนจำนวนเดิม และหักจำนวนจาก Inventory ใหม่ใน Transaction

4.3 Activity การจองการรด์

Initial Node คือจุดเริ่ม Action คือกิจกรรมที่ดำเนินการ Decision ใช้เงื่อนไขแยกทาง และ Final Node คือจุดสิ้นสุด
เส้นทางสต็อกไม่พอจบด้วยข้อผิดพลาด ส่วนเส้นทางสำเร็จรวมราคาหลัง Loop ตรวจรายการครบแล้ว

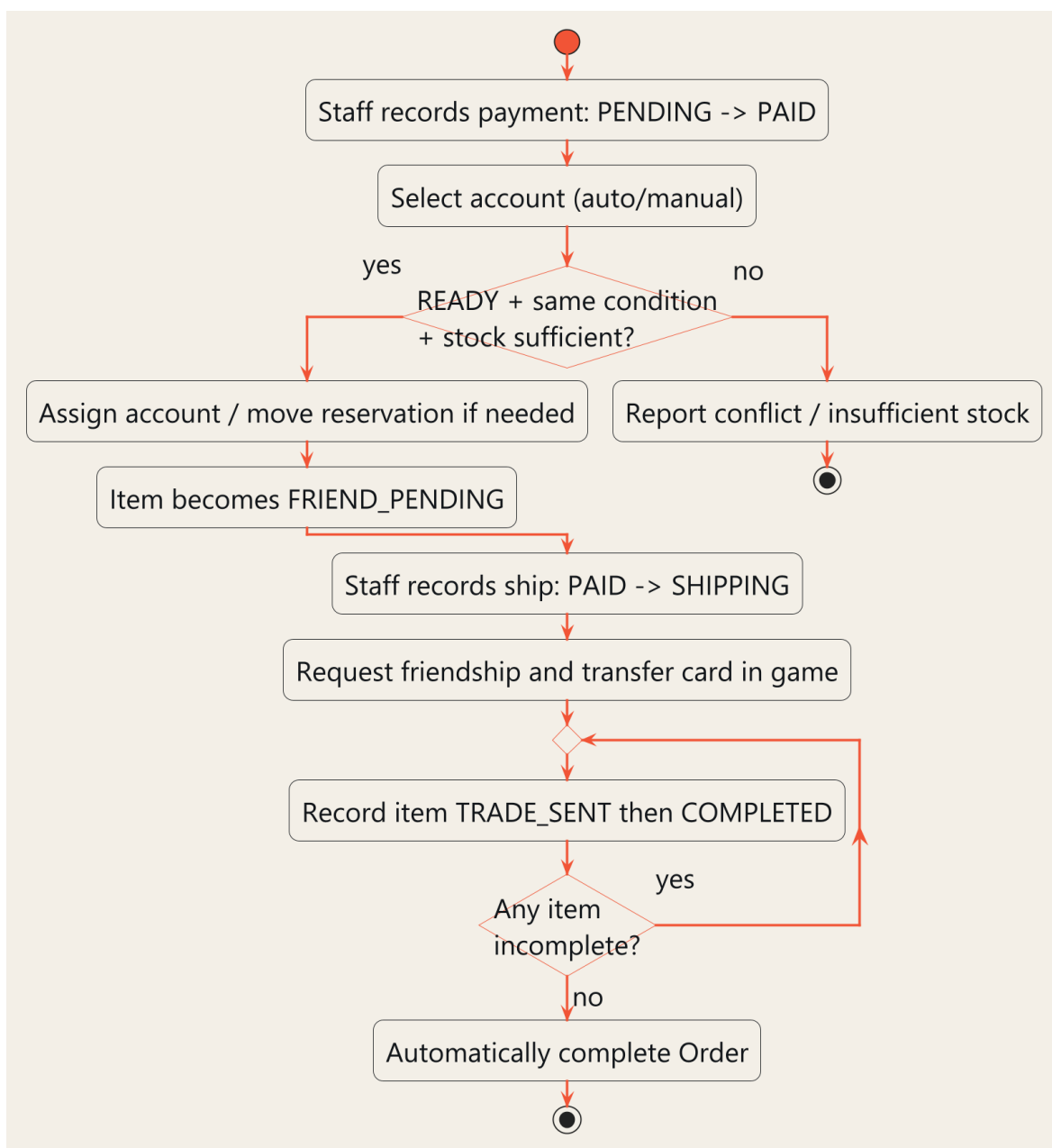


ภาพที่ 14 Activity Diagram การสร้างคำสั่งซื้อ

การตรวจ ownership และการย้อนกลับเมื่อ Item ใดล้มเหลวเป็นเงื่อนไขร่วมของ Service ซึ่งมีหลักฐานใน OrderAccessPolicy และชุด OrderStockAndSecurityTest

4.4 Activity การเทรด

Activity นี้แยกสถานะคำสั่งซื้อออกจากการส่งมอบราย Item แสดงการตัดสินใจเมื่อบัญชีไม่พร้อมหรือ Items ยังไม่ครบ ไม่ควรใช้เพียงการกดปุ่มบน UI เป็นหลักฐานว่า Order สำเร็จ หาก Server ยังไม่ผ่าน Guard

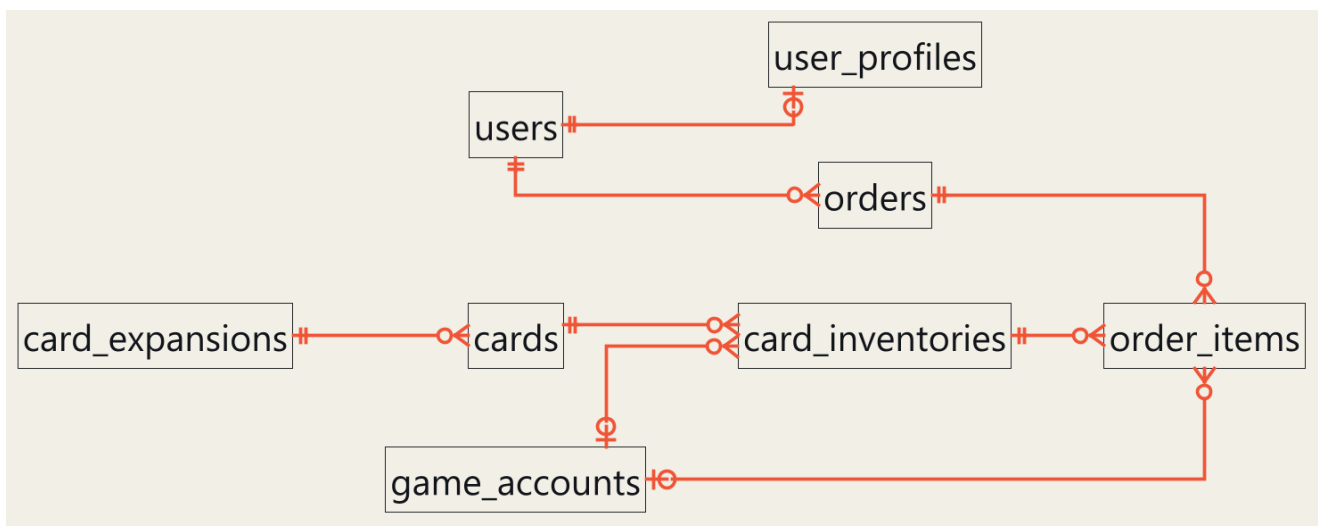


ภาพที่ 15 Activity Diagram การส่งมอบและตรวจครบทุก Item

การทำงานในเกมเป็นกิจกรรมของร้านและลูกค้า พนักงานจึงบันทึกผลกลับเข้าระบบ เมื่อ Item สุดท้าย COMPLETED แล้ว UI ควรโหลดหรือใช้ข้อมูล Order ล่าสุดเพื่อแสดงว่าคำสั่งซื้อปิดอัตโนมัติ

บทที่ 5 การออกแบบฐานข้อมูล

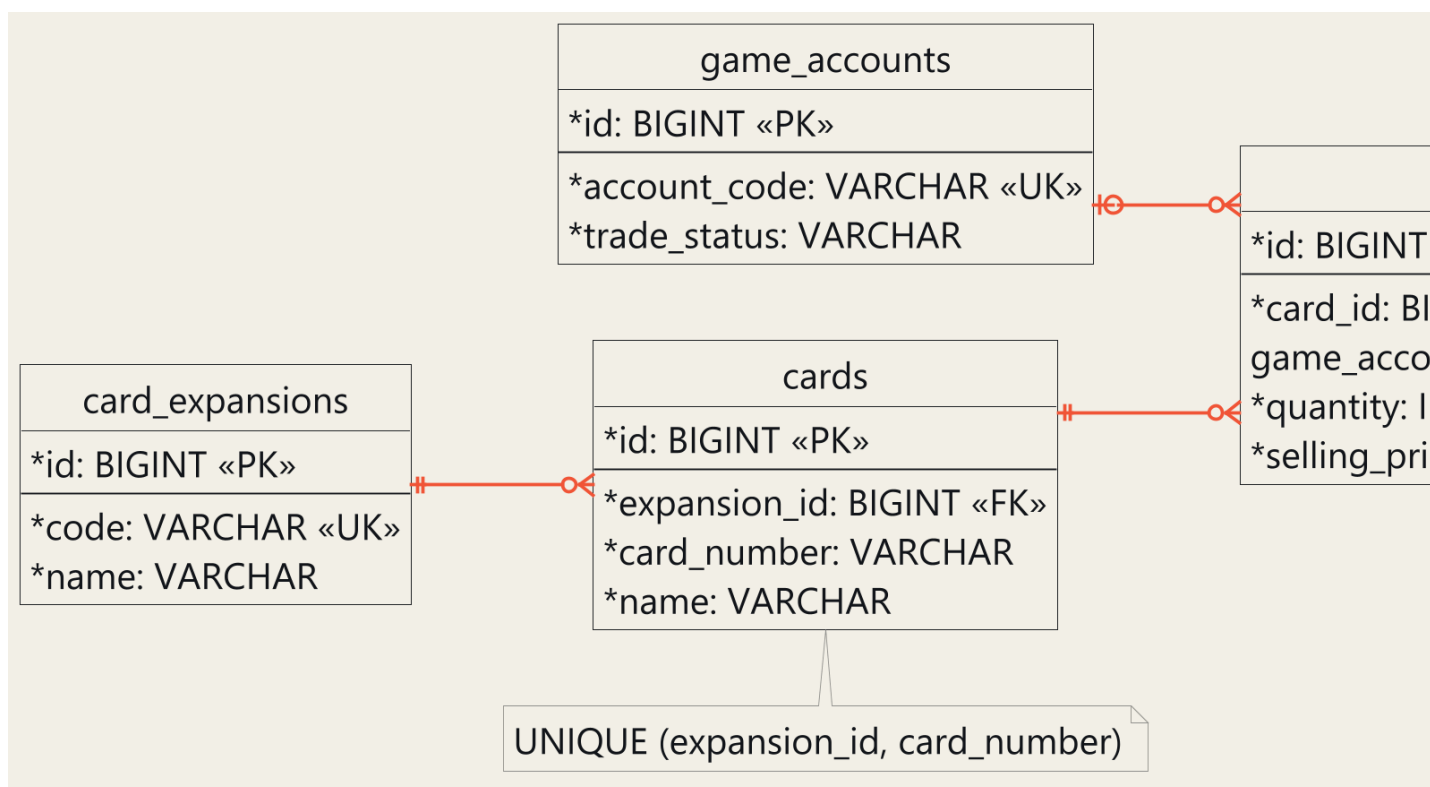
ฐานข้อมูลเชิงสัมพันธ์มี 8 ตาราง แยกข้อมูลผู้ใช้และโปรไฟล์ ข้อมูลชุดการ์ดและการ์ด คลังบัญชีเกมและสต็อก และคำสั่งซื้อกับรายการส่งมอบ ER ใช้ Crow's Foot โดยอิง Nullability และ Constraints ใน schema.sql จึงเป็นแผนภาพฐานข้อมูล ไม่ใช่ UML Class Diagram



ภาพที่ 16 ER Diagram ภาพรวมทั้ง 8 ตาราง

users → user_profiles เป็น 1 ต่อ 0..1 ด้วย UNIQUE(user_id) ส่วน users → orders expansion → cards
 cards → inventories และ orders → order_items เป็น 1 ต่อหลาย game_account_id และ assigned_account_id
 เป็น Nullable FK ทำให้รายการข้อมูลหนึ่งรายการไม่มี Account อ้างอิงก็ได้ ไม่มีความสัมพันธ์ Many to Many ใน Schema นี้

5.1 ER ของ Catalog และ Vault

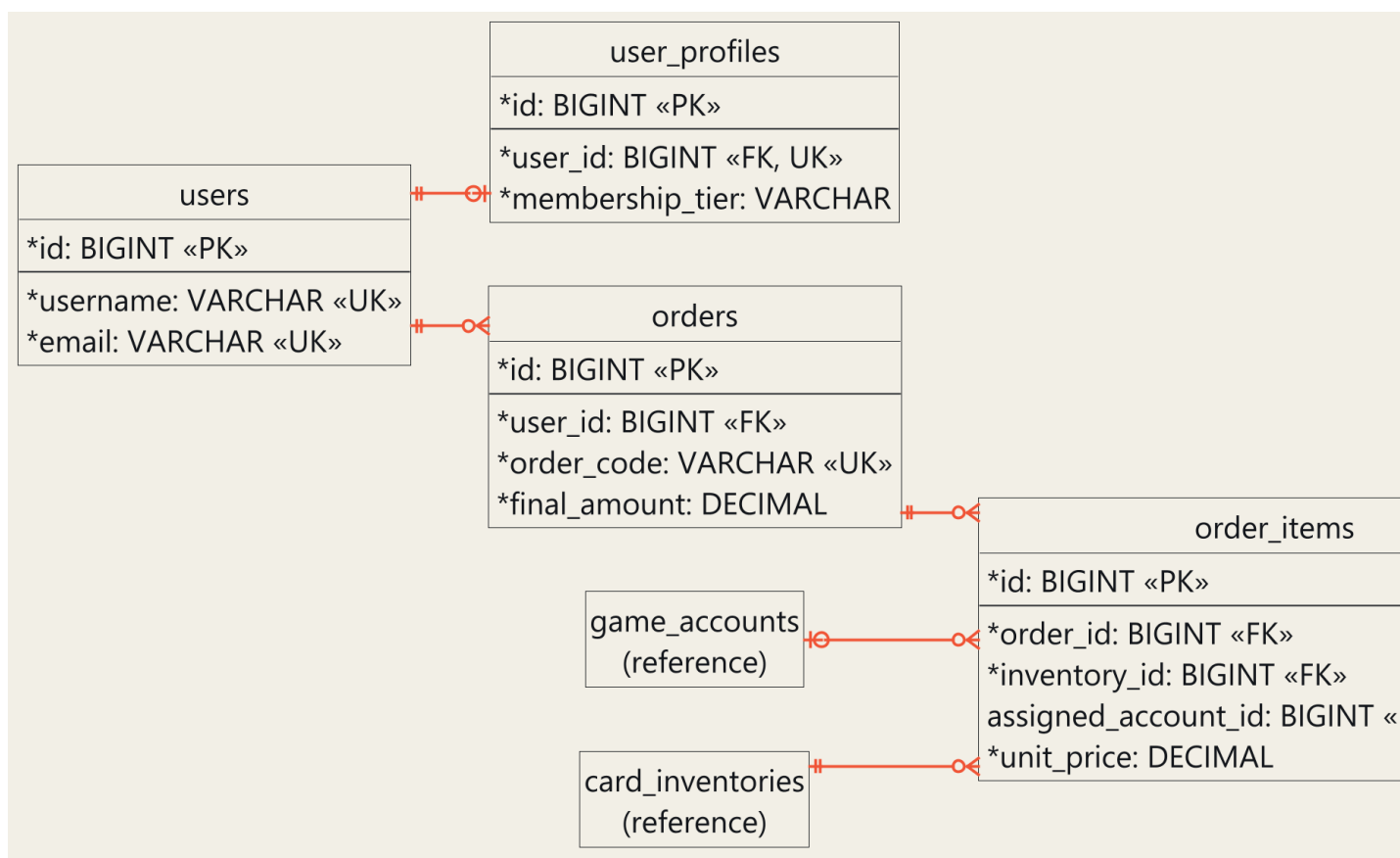


ภาพที่ 17 ER Diagram กลุ่มการ์ด บัญชีเกม และสต็อก

cards ใช้ **expansion_id** อ้างอิงชุดการ์ดและห้ามเลขการ์ดซ้ำภายในชุดเดียวกันด้วย **UNIQUE(expansion_id, card_number)** **card_inventories** เก็บจำนวนและราคาตาม Card Condition และบัญชีเกม Account FK เลือกเป็น NULL ได้ และเมื่อบัญชีถูกลบตามกฎ SQL FK ใช้ SET NULL ส่วน Card ที่ยังถูก Inventory อ้างอิงใช้ RESTRICT

GameAccountService ป้องกันการลบบัญชีที่ยังมี Inventory หรือ Order References เพิ่มเติมในระดับแอปพลิเคชัน แม้ DDL มี SET NULL จึงต้องอ่าน Business Guard ประกอบ SQL Delete Behavior ไม่ใช่ SQL Constraint เพียงอย่างเดียว อธิบายสิทธิ์การลบผ่าน API

5.2 ER ของ User และ Order



ภาพที่ 18 ER Diagram กลุ่มผู้ใช้และคำสั่งซื้อ

Profile เป็นข้อมูลประกอบของ User ที่มีได้ไม่เกินหนึ่งรายการ OrderItem เก็บจำนวน unit_price และ subtotal ณ เวลาจอง เพื่อให้ประวัติคำสั่งซื้อไม่เปลี่ยนแปลงตามราคาหน้าคลัง OrderItem อ้างอิง Inventory และบัญชีที่ Assign ให้ส่งมอบ โดยใช้ FK คนละคอลัมน์

การลบ User ไม่ทำให้ Orders ที่อ้างอิงหายไป เพราะ order.user_id เป็น RESTRICT ส่วนการลบ Order ใช้ CASCADE ไปยัง OrderItems ราคาทั้งหมดใช้ DECIMAL(10,2) และกฎไม่ให้ติดลบถูกกำหนดด้วย CHECK ใน DDL

5.3 Foreign Key Cascade และ Fetch

ตารางที่ 11 กฎ SQL และ JPA ที่ต้องอ่านแยกกัน

Relation	SQL ON DELETE	JPA Fetch และ Cascade
User → Profile	CASCADE	Profile LAZY; ALL; orphanRemoval
User → Orders	RESTRICT ฝั่ง Order.user	Order.user LAZY
Expansion → Cards	RESTRICT ฝั่ง Card.expansion	Card.expansion EAGER; Cards LAZY และ ALL
Card → Inventories	RESTRICT	CardInventory.card LAZY
Account → Inventories	SET NULL	CardInventory.gameAccount LAZY
Order → Items	CASCADE	Order.items LAZY; ALL; orphanRemoval
Inventory → Items	RESTRICT	OrderItem.inventory LAZY
Account → Assigned Items	SET NULL	OrderItem.assignedAccount LAZY

SQL ON DELETE ทำงานที่ฐานข้อมูลเมื่อมีการลบแถวที่เป็น Parent ส่วน JPA Cascade ส่งต่อ Operation ของ EntityManager เช่น persist และ remove ตาม Mapping ไม่ใช่กลไกเดียวกัน orphanRemoval ใช้ลบ Child ที่ถูกนำออกจากความสัมพันธ์ของ Parent ที่เป็นเจ้าของ

เหตุผลเลือก Fetch

ความสัมพันธ์หลายรายการส่วนใหญ่เป็น LAZY เพื่อโหลดเมื่อใช้ ลดการดึงกราฟข้อมูลทั้งระบบใน Request ทั่วไป Card.expansion เป็น EAGER ตาม Mapping ปัจจุบัน เพราะข้อมูลการ์ดมักแสดงชุดการ์ดด้วย จึงไม่ได้อ้างว่าทุกความสัมพันธ์เป็น LAZY

```

57 @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
58 @Builder.Default
59 private List<OrderItem> items = new ArrayList<>();
60

```

Order.addItem กำหนดความสัมพันธ์สองทางให้ Item อ้าง Order ก่อนบันทึก จึงใช้ Order เป็น Aggregate Owner สำหรับการสร้างรายการ ไม่เปิด API ให้ลูกค้าบันทึก OrderItem ที่ไม่มี Order

5.4 Constraints ดัชนี และ Migration

Constraints และดัชนีที่มีใน DDL

ทุกตารางมี BIGSERIAL Primary Key รวม 8 รายการ มี Unique Constraints รวม 7 รายการ ได้แก่ username email profile.user_id expansion.code คู่ expansion_id กับ card_number account_code และ order_code PostgreSQL สร้างดัชนีสำหรับ Primary Key และ Unique Constraint ให้ ส่วน DDL ปัจจุบันไม่มี CREATE INDEX แยกสำหรับ Foreign Key หรือการกรองอื่น

ตารางที่ 12 Constraints ที่ใช้รักษาความถูกต้อง

กฎ	จุดใช้งาน	เหตุผล
UNIQUE	users.username / users.email	ห้ามบัญชีซ้ำ
UNIQUE	user_profiles.user_id	บังคับ Profile ไม่เกินหนึ่งต่อ User
UNIQUE แบบคู่	cards(expansion_id, card_number)	เลขซ้ำได้ข้ามชุด แต่ไม่ซ้ำในชุดเดียวกัน
CHECK	quantity และราคาของ Inventory / Item / Order	รักษาจำนวนและจำนวนเงินตามช่วงที่กำหนด
FOREIGN KEY	ความสัมพันธ์ระหว่างทั้ง 8 ตาราง	ป้องกันข้อมูลอ้างอิงที่ไม่มี Parent

หากข้อมูลเพิ่มขึ้น จุดที่ควรประเมินดัชนีเพิ่มเติมคือ orders.user_id order_items.order_id และการค้น Inventory ตาม card_id account_id และ condition ต้องวัด Query Plan และพฤติกรรมโหลดก่อนเพิ่ม ไม่อ้างว่าดัชนีเหล่านี้มีอยู่แล้วหรือว่ามีผล Benchmark รองรับ

Migration Script

มี core-schema-alignment.sql สำหรับปรับชนิดและ Nullability ของฐานข้อมูลเดิม และ a1a-local-images.sql สำหรับอัปเดตเลขการ์ดกับ image_url ของ A1a โดยคง Card IDs และ References ทั้งสองเป็น Manual PostgreSQL Migration มี BEGIN และ COMMIT ให้การเปลี่ยนแปลงอยู่ใน Transaction ไม่ใช่ Flyway หรือ Liquibase Migration ที่รันอัตโนมัติ

core-schema-alignment ตรวจสอบค่าที่เกินความยาวหรือไม่ผ่านสัญญา ก่อน ALTER และยุติเมื่อพบปัญหา ส่วน a1a-local-images ใช้ WHERE จำกัดชุด ชื่อ และเลขเดิม/ใหม่ จึงไม่อัปเดตทุกการ์ด การใช้งานกับฐานข้อมูลจริงควรสำรองก่อนและสั่ง psql โดยเปิด ON_ERROR_STOP

5.5 หลักการจัดเก็บและ Transaction

ข้อมูลหลักและข้อมูล ณ เวลาทำรายการ

Card เก็บตัวตนของการ์ด เช่นชื่อ ชุด เลข ความหายาก และ URL รูปภาพ ส่วน CardInventory เก็บข้อมูลที่เปลี่ยนตามบัญชีและสภาพ เช่นจำนวน ราคาทุน และราคาขาย OrderItem เก็บ Unit Price และ Subtotal ในเวลาจอง การออกแบบนี้ลดการซ้ำของข้อมูลการ์ดและเก็บประวัติราคาให้คำสั่งซื้อ

Reservation และการเลือกบัญชี

ระบบหัก Inventory.quantity เมื่อสร้าง Order เพื่อกันสต็อกให้การจอง เมื่อเปลี่ยนบัญชีสำหรับ Item จะคืนจำนวนจาก Inventory เดิมและหักจาก Inventory เป้าหมาย หากจับคู่กลับไปยัง Inventory ที่ถือ Reservation เดิมจำนวนที่จองแล้วถูกนับใน Effective Stock เพื่อไม่ให้เกิดการจองใบสุดท้ายถูกมองว่าของหมด

การยกเลิกและการย้อนกลับ

ยกเลิกจากสถานะที่อนุญาตจะคืน Stock ของ Items การสร้างหลาย Item อยู่ใน Transaction จึงควรย้อนกลับทั้งหมดเมื่อบาง Item มีปัญหา หลักฐานทดสอบที่เกี่ยวข้องคือ MultiItemRollbackTests CancelOrderRestorationTests และ LastCardMatchingTests ใน OrderStockAndSecurityTest

สภาพแวดล้อมฐานข้อมูล

Profile local ใช้ H2 แบบ In Memory สำหรับการพัฒนา ข้อมูลไม่ใช่พื้นที่จัดเก็บถาวร Profile prod ใช้ PostgreSQL โดยอ่าน Connection URL Username และ Password จาก Environment Variables ข้อมูลใช้งานจริงจึงอยู่ใน Neon แยกจาก Filesystem ของ Render

การเริ่มต้นมี Hibernate ddl-auto:update และ SQL initialization ซึ่งต่างจากระบบ Versioned Migration เต็มรูปแบบ prod ตั้ง continue-on-error เพื่อข้าม Seed Insert ที่ชนข้อมูลเดิม แต่ควรติดตาม Log และจัดการ Migration อย่างมีขั้นตอนเมื่อ Schema เปลี่ยน

ขอบเขตการรับรอง

Transaction และ Test ของกรณี Rollback ไม่ได้เป็นหลักฐานว่าป้องกันทุกกรณี Concurrent Booking หรือ Deadlock แล้ว ระบบปัจจุบันยังควรเพิ่มการทดสอบหลาย Request พร้อมกันและพิจารณา Lock หรือ Version Field ตามปริมาณงานจริง

บทที่ 6 REST API และส่วนติดต่อผู้ใช้

REST API ใช้เส้นทาง /api/v1 เป็น Version Prefix และใช้ Resource Names เช่น cards accounts และ orders CRUD ครบสอง Resources คือ Cards และ Game Accounts โดย POST สร้างข้อมูล PUT แก้ไขข้อมูล GET อ่านข้อมูล และ DELETE ลบข้อมูลตาม Guard ที่กำหนด

ตารางที่ 13 HTTP Methods และ Status Codes

HTTP	ผลสำเร็จหลัก	ความหมาย
GET	200 OK	อ่านรายการหรือ Resource
POST สร้าง Resource	201 CREATED	สร้าง Card Account Order หรือบันทึก Pull
PUT	200 OK	แก้ไขรายละเอียด Resource
PATCH	200 OK	เปลี่ยนสถานะ ราคา หรือข้อมูลบางส่วน
DELETE	204 NO CONTENT	ลบสำเร็จโดยไม่มี Response Body

สัญญาของผลสำเร็จ

ApiResponse<T> ประกอบด้วย success message data และ timestamp Resource ที่คืนข้อมูลใช้ DTO เช่น CardResponse และ GameAccountResponse ส่วน DELETE คืน ResponseEntity<Void> จึงไม่มี JSON Body ตัวอย่างด้านล่างเป็นรูปแบบสัญญา ไม่ใช่การบันทึกผลจาก Request ที่สร้างข้อมูลใหม่ใน Production

```
{
  "success": true,
  "message": "Success",
  "data": { "id": 1, "name": "Mew ex" },
  "timestamp": "2026-10-10T14:00:00"
}
```

เอกสาร API

OpenAPI อธิบาย Paths Methods Parameters Request Schemas และ Responses ส่วน Swagger UI ใช้แสดงและทดลอง Request จากสัญญาเดียวกัน ปัจจุบันหน้า Production แสดง OAS 3.1 และ Generated Server URL เป็น HTTPS ของ Render

6.1 CRUD ของ Cards

ตารางที่ 14 Endpoints ของ Card Catalog

Method และ URL	งานที่ทำ	ผลสำเร็จ
POST /api/v1/cards	เพิ่มข้อมูลการ์ดจาก CardRequest	201
GET /api/v1/cards	อ่านรายการและตัวกรอง	200
GET /api/v1/cards/{id}	อ่านการ์ดหนึ่งรายการ	200
PUT /api/v1/cards/{id}	แก้ไขรายละเอียดการ์ด	200
DELETE /api/v1/cards/{id}	ลบการ์ดที่ลบได้ตามข้อมูลอ้างอิง	204
GET /api/v1/cards/paged	อ่านแบบแบ่งหน้าและเรียงลำดับ	200
GET /api/v1/cards/search?q=...	ค้นหาชื่อการ์ด	200
GET /api/v1/cards/expansions	อ่านชุดการ์ด	200

ตัวอย่าง Request สำหรับสร้างหรือแก้ไข

```
{
  "expansionId": 2,
  "cardNumber": "003/068",
  "name": "Celebi ex",
  "cardType": "POKEMON",
  "rarity": "DOUBLE_RARE",
  "elementType": "GRASS",
  "hp": 130,
  "retreatCost": 1,
  "imageUrl": "/images/cards/A1a_003_EN.webp"
}
```

expansionId ต้องอ้างอิงถึงข้อมูลที่มีอยู่จริง และคู่ expansionId กับ cardNumber ต้องไม่ซ้ำ การอ่าน Card เป็นสาธารณะ ส่วน POST PUT และ DELETE ต้องมีบทบาท STAFF หรือ ADMIN ตัวอย่างใช้ค่าที่มีโครงสร้างตรง DTO แต่การสาธิต CRUD ควรใช้ Resource ใหม่เพื่อไม่แก้ไขข้อมูล Seed ที่อ้างอิงกับ Inventory

6.2 CRUD ของ Game Accounts

ตารางที่ 15 Endpoints ของ Game Account Vault

Method และ URL	งานที่ทำ	ผลสำเร็จ
POST /api/v1/accounts	สร้างบัญชีเกมของร้าน	201
GET /api/v1/accounts	อ่านบัญชีเกมทั้งหมด	200
GET /api/v1/accounts/{id}	อ่านบัญชีเกมหนึ่งรายการ	200
PUT /api/v1/accounts/{id}	แก้รายละเอียดบัญชี	200
DELETE /api/v1/accounts/{id}	ลบบัญชีที่ไม่มี References ตาม Guard	204
POST /api/v1/accounts/{id}/pulls	เพิ่มการ์ดที่เปิดของได้	201
GET /api/v1/accounts/{id}/cards	อ่านการ์ดในบัญชี	200
PATCH /api/v1/accounts/{id}/trade-status	ตั้งค่าความพร้อมเทรด	200

ตัวอย่าง GameAccountRequest

```
{
  "accountCode": "PV-DEMO-01",
  "inGameName": "DemoVault",
  "friendId": "1234-5678-9012-3456",
  "tradeStatus": "READY",
  "buyInCost": 0.00,
  "notes": "Demo account"
}
```

บัญชีเกมกับบัญชีผู้ใช้เว็บไซต์เป็นคนละ Resource friendId ต้องเป็นรหัส 16 หลัก รูปแบบมีหรือไม่มีขีดตาม Validation ใน DTO accountCode ไม่ซ้ำ และต้นทุนต้องไม่ติดลบ Endpoints ของ Accounts ใช้ได้โดย STAFF หรือ ADMIN รายการ Accounts ปัจจุบันคืน List ไม่ได้มี Pagination เหมือน Cards/paged

6.3 Orders และ Trade Matching

ตารางที่ 16 Endpoints ในการจองและส่งมอบ

Method และ URL	ข้อมูลเพิ่ม	หน้าที่
POST /api/v1/orders	PlaceOrderRequest	สร้างการจอง 201
GET /api/v1/orders/{id}	Order ID	อ่านออเดอร์ที่มีสิทธิ์ 200
PATCH /api/v1/orders/{id}/status	action=pay / ship / complete / cancel	เรียก State Action 200
PATCH /api/v1/orders/{orderId}/items/{itemId}/trade-status	status=TRADE_SENT / COMPLETED	บันทึกผลส่งมอบ 200
GET /api/v1/trades/orders/{orderId}/recommendations	Order ID	อ่านบัญชีที่แนะนำ 200
POST /api/v1/trades/orders/{orderId}/auto-match	Order ID	Assign ทุก Item 200
POST /api/v1/trades/items/{itemId}/assign	accountId	Assign ด้วยพนักงาน 200

ตัวอย่าง PlaceOrderRequest

```
{
  "userId": 3,
  "customerFriendId": "1234-5678-9012-3456",
  "customerInGameName": "RedMaster",
  "items": [ { "inventoryId": 1, "quantity": 1 } ],
  "notes": "Please confirm in chat"
}
```

Request ไม่กำหนดราคาหรือเปอร์เซ็นต์ส่วนลด ลูกค้าสร้างได้เฉพาะ userId ของตน ส่วนพนักงานสร้างแทนลูกค้าได้ Server ใช้ราคา Inventory และ Tier จาก Profile เพื่อคำนวณคำสั่งซื้อ เมื่อส่ง Item ครบแล้ว Order ปิดอัตโนมัติ จึงไม่จำเป็นต้องเรียก complete ขึ้น

itemId ในตารางหมายถึง orderItemId ของ API จริง และต้องเป็น Item ที่อยู่ภายใต้ orderId นั้น API ตรวจสอบ Item ภายใน Order เพื่อป้องกันการอ้าง Item ข้ามคำสั่งซื้อ

6.4 Validation และ Global Exception Handler

Controller ใช้ @Valid ที่ Request Body ส่วน DTO กำหนด @NotNull @NotBlank @Size @Min และ @Pattern เช่นชื่อ Card ยาวไม่เกิน 100 ตัวอักษร เลข Card ไม่เกิน 20 และ Friend ID มี 16 หลัก Nested Items ใช้ @Valid เพื่อให้ตรวจสอบแต่ละรายการด้วย

```

21 @NotNull(message = "Expansion ID is required")
22 private Long expansionId;
23
24 @NotBlank(message = "Card number is required")
25 @Size(max = 20, message = "Card number must not exceed 20 characters")
26 private String cardNumber;
27
28 @NotBlank(message = "Card name is required")
29 @Size(max = 100, message = "Card name must not exceed 100 characters")
30 private String name;

```

ตารางที่ 17 สัญญาข้อผิดพลาด

HTTP	Error code หรือกรณี	ผู้จัดการ
400	VALIDATION_FAILED / INVALID_PARAMETER / INSUFFICIENT_STOCK	GlobalExceptionHandler
401	AUTHENTICATION_REQUIRED	Security Filter Chain
403	ACCESS_DENIED / ข้ามเจ้าของข้อมูล	Security และ Exception Handler
404	Resource ที่ร้องขอไม่มี	GlobalExceptionHandler
409	STATE_CONFLICT / สถานะหรือข้อมูลขัดแย้ง	GlobalExceptionHandler
500	INTERNAL_SERVER_ERROR ที่ไม่คาดหมาย	GlobalExceptionHandler

```

{
  "timestamp": "2026-10-10T14:00:00",
  "status": 400,
  "error": "VALIDATION_FAILED",
  "message": "Validation error in request payload",
  "path": "/api/v1/cards",
  "details": { "name": "Card name is required" }
}

```

ตัวอย่างเป็นโครงสร้าง ErrorResponse การแสดง Validation ราย Field ช่วยให้ UI บอกผู้ใช่ว่าต้องแก้ช่องใด HTTP 500 ไม่ใช่สถานะสำหรับความผิดพลาดทุกชนิด และควรใช้ Log เพื่อวิเคราะห์กรณีไม่คาดหมาย ใน Production ควรปรับข้อความ General Exception ให้ไม่เผยรายละเอียดภายในโดยไม่จำเป็น

6.5 Pagination Sorting และ Filtering

GET /api/v1/cards/paged รับ Pageable ของ Spring Data และ CardFilterRequest ค่า Default คือ page=0 size=10 sort=cardNumber แบบ ASC Service สร้าง Specification สำหรับชื่อ ชุด ความหายาก ประเภท และธาตุ แล้วคืน PageResponse ที่มี Metadata

```
GET /api/v1/cards/paged
?page=0&size=10&sort=name,asc
&expansionCode=A1a&rarity=DOUBLE_RARE
```

ตารางที่ 18 พารามิเตอร์การแบ่งหน้าและตัวกรอง

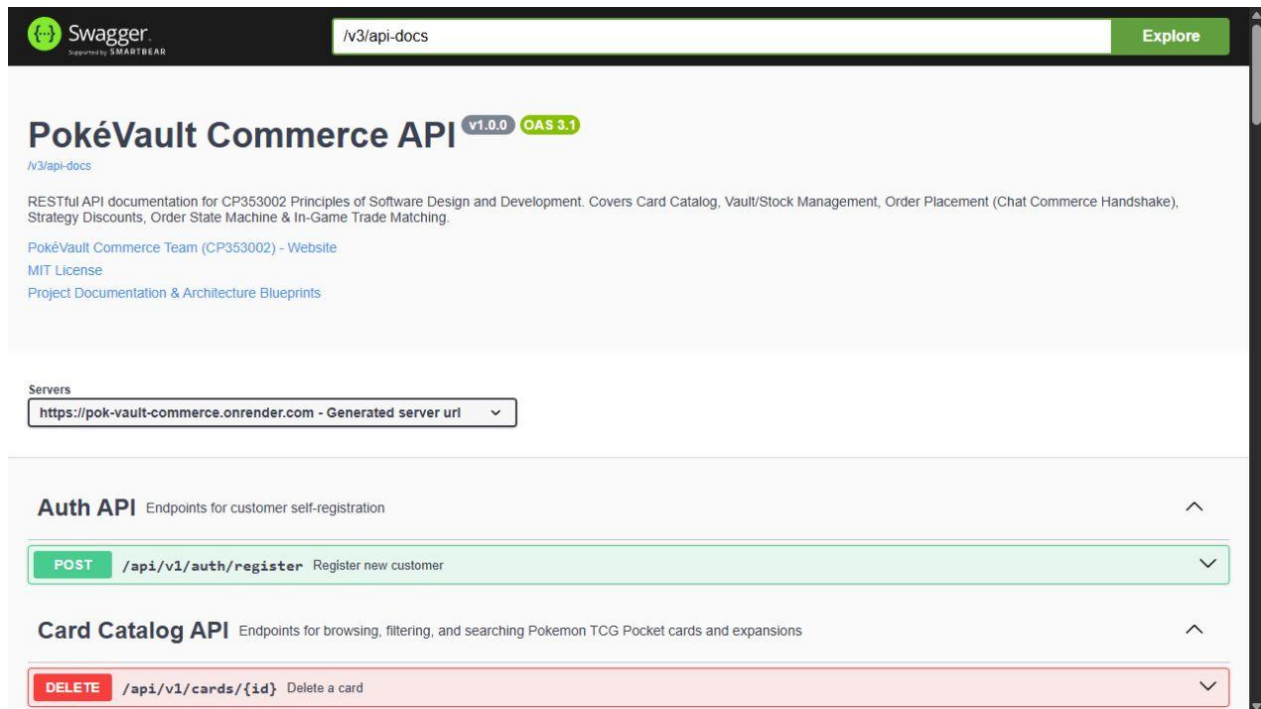
Parameter	ความหมาย	ตัวอย่าง
page	ลำดับหน้าเริ่มจาก 0	0
size	จำนวนรายการต่อหน้า	10
sort	Property และทิศทางเรียง	name,asc
name	ชื่อที่ใช้กรอง	Mew
expansionCode	รหัสชุด	A1a
rarity	ระดับความหายาก	DOUBLE_RARE
cardType	ประเภทการ์ด	POKEMON
elementType	ธาตุ	GRASS

PageResponse ประกอบด้วย content pageNumber pageSize totalElements totalPages isFirst และ isLast เพื่อให้ UI สร้างการนำทางได้โดยไม่คำนวณจากจำนวนรายการที่เห็นเพียงหน้าเดียว การเรียงลำดับทำที่ Query ก่อนแบ่งหน้า

Filter enum ที่ไม่ถูกต้องใน CardServiceImpl ถูกข้ามในบางแขนงด้วย catch IllegalArgumentException จึงไม่อ้างว่าทุกค่าตัวกรองผิดรูปแบบจะตอบ 400 เหมือน Request Body จุดนี้ควรปรับสัญญา Query Parameters ให้ชัดเจนเมื่อขยายระบบ ส่วน Game Accounts ปัจจุบันยังคืน List

6.6 Swagger UI บนเว็บไซต์จริง

ภาพต่อไปนี้เก็บจาก URL Production วันที่ 10 ตุลาคม 2569 Swagger แสดงกลุ่ม Card Catalog API และ Server URL เป็น HTTPS ของ Render พร้อม Paths และ HTTP Methods จาก Application ที่ Deploy แล้ว



ภาพที่ 19 Swagger UI จากบริการที่ Deploy บน Render

Swagger UI: <https://pok-vault-commerce.onrender.com/swagger-ui/index.html>

OpenAPI JSON: <https://pok-vault-commerce.onrender.com/v3/api-docs>

ขั้นตอนอ่านและใช้งาน

- เลือก Tag ของ Resource และ Method ที่ต้องการ
- อ่าน Parameters และ Schema ก่อนใช้ Try it out
- งานเขียนข้อมูลต้องเข้าสู่ระบบด้วย STAFF หรือ ADMIN ใน Session เดียวกัน
- GET สาธารณะใช้ดูข้อมูลได้ ส่วนการทดลองสร้างและลบให้ใช้ข้อมูล Demo แยกจาก Orders ที่กำลังทำงาน
- ตรวจ HTTP Status และ Response Body เพื่อแยก Validation สิทธิ์ และ Business Conflict

ภาพหน้าจอยืนยันว่า Swagger เปิดได้ในเวลาที่เก็บภาพ แต่ไม่ได้เป็นหลักฐานว่าทุก Endpoint ถูกทดสอบผ่าน Production แล้ว การทดสอบ API เชิงพฤติกรรมอยู่ในบทที่ 7 และภาคผนวกผลทดสอบ

6.7 Security และการป้องกันข้อมูลข้ามบัญชี

Spring Security ใช้ Form Login Session และ BCrypt PasswordEncoder โดย Visitor อ่านหน้าแรกและ Cards ได้ CUSTOMER สร้างและอ่าน Orders ของตน ส่วน STAFF และ ADMIN เข้าหน้าจัดการ Inventory Accounts Orders และ APIs สำหรับงานบริหาร

ตารางที่ 19 ขอบเขตสิทธิ์การใช้งาน

งาน	Visitor	CUSTOMER	STAFF / ADMIN
อ่าน Catalog	ได้	ได้	ได้
จองการ์ด	เข้าสู่ระบบก่อน	เฉพาะตนเอง	จองแทนลูกค้าได้
อ่าน Order	ไม่ได้	เฉพาะตนเอง	ได้
แก้ Card และ Account	ไม่ได้	ไม่ได้	ได้
เปลี่ยน Order / Item Status	ไม่ได้	ไม่ได้	ได้
เปลี่ยนราคาและ Tier	ไม่ได้	ไม่ได้	ได้

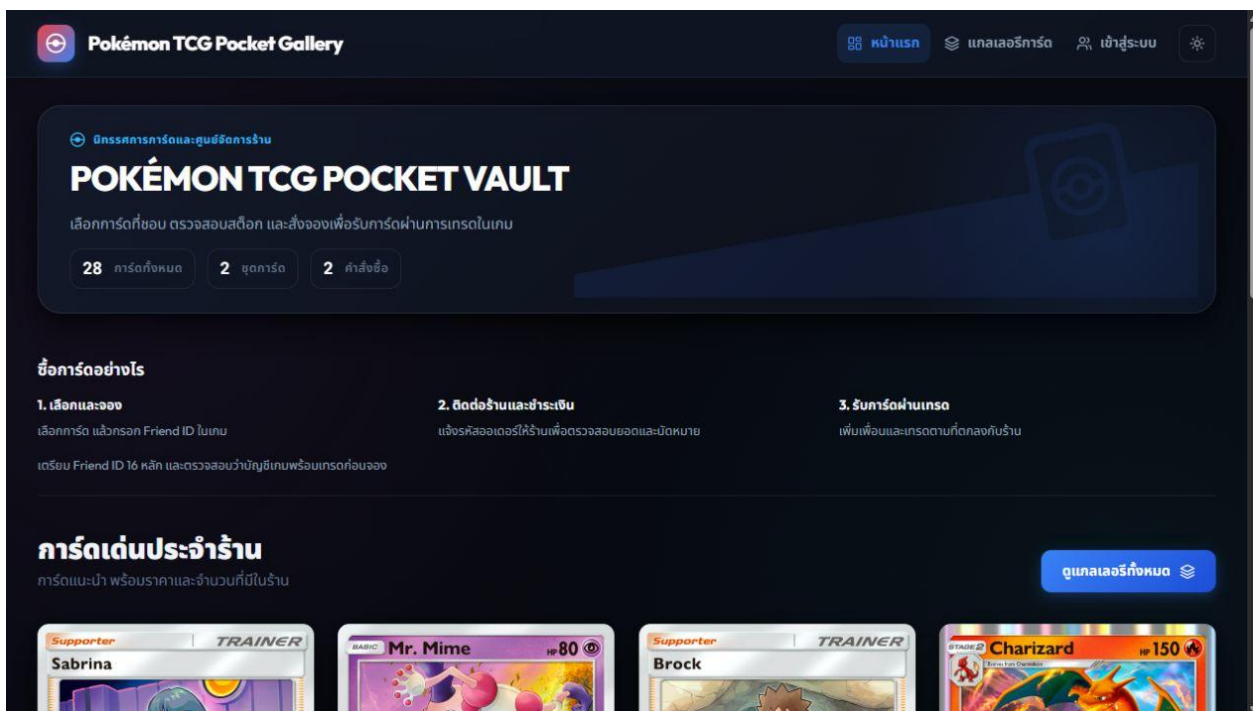
Ownership และ Defense in Depth

OrderAccessPolicy ตรวจสอบสิทธิ์ที่ @PreAuthorize ของ Controller และ OrderService ตรวจสอบผู้ใช้จาก SecurityContext ร่วมกับ Order Owner การตรวจที่ Service ช่วยให้ไม่เชื่อ userId จาก Request โดยล้าพัง อีกทั้งการอัปเดต Item คั่น Item ภายใน Order ที่ระบุ

ขอบเขตที่ควรพัฒนาต่อ

SecurityConfig ยกเว้น CSRF สำหรับ /api/** และใช้ Session Authentication จึงควรทบทวนการป้องกัน CSRF หากใช้งานจริงนอกบริบท Demo นอกจากนี้ควรแยกบัญชี Seed ออกจากผู้ใช้งานจริง จำกัด Actuator ที่จำเป็น และไม่เปิด H2 Console ใน Production การเพิ่ม JWT ไม่ใช่เงื่อนไขของงานนี้และยังไม่มีในระบบ

6.8 หน้าแรกและการเริ่มซื้อ



ภาพที่ 20 หน้าแรกของเว็บที่ Deploy แล้ว

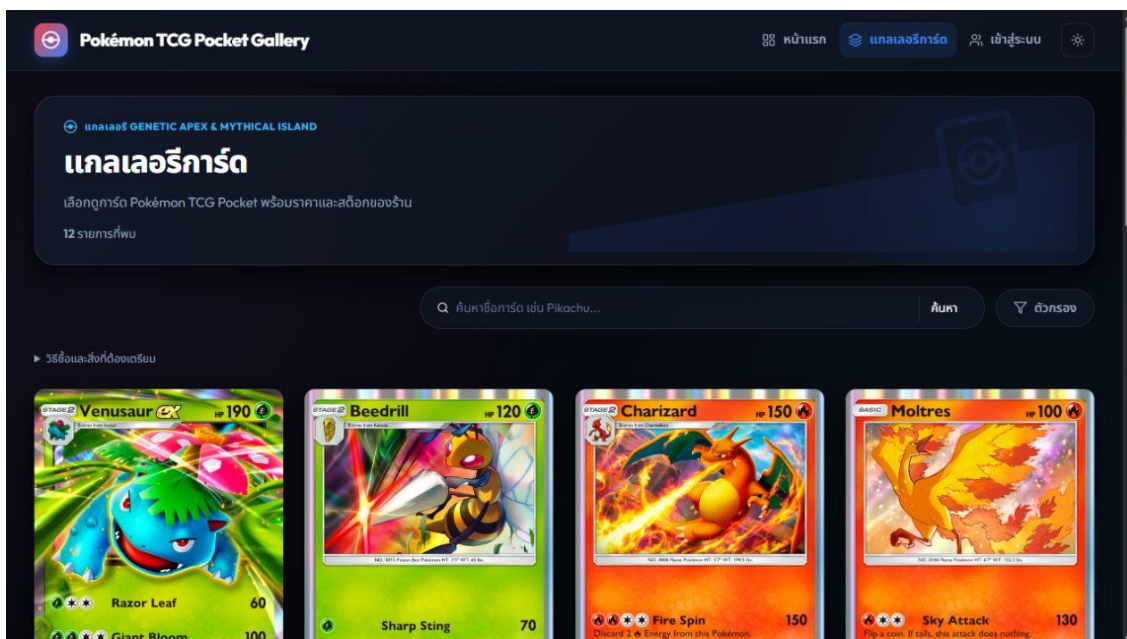
หน้าแรกอธิบายขั้นตอนเลือกและจอง ติดต่อร้านเพื่อชำระเงิน และรับการ์ดผ่านการเทรด แสดง Friend ID ที่ต้องเตรียมไว้ก่อนจอง พร้อมรายการการ์ดแนะนำที่มีราคาและสต็อก ผู้ใช้จึงเข้าใจขอบเขตงานของเว็บตั้งแต่ก่อนทำการ

แนวทาง UX ที่นำมาใช้

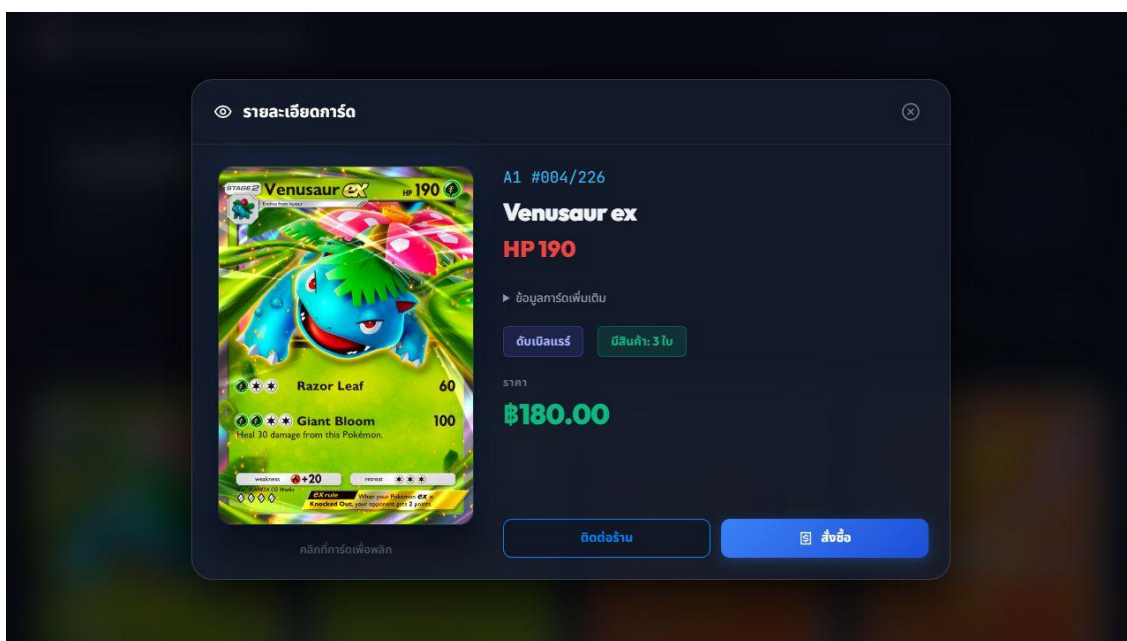
- แสดงงานหลักและขั้นตอนซื้อในภาษาของผู้ใช้
- แสดงราคาและจำนวนคงเหลือคู่กับการ์ดก่อนเปิดรายละเอียด
- จัดเมนูตามบทบาท เพื่อให้พนักงานเห็นหน้าจัดการและลูกค้าเห็นงานที่ใช้
- ใช้ข้อความสถานะประกอบสี เพื่อให้การสื่อสารไม่ขึ้นกับสีเพียงอย่างเดียว

ภาพหน้าจอเป็นข้อมูลในเวลาที่เกิดภาพ จำนวน Cards หรือ Orders เปลี่ยนตามข้อมูลในระบบ ไม่ใช่ตัวเลขทดสอบที่ใช้รับรองการทำงาน ภาพหน้าแรกและภาพใหม่ในส่วนนี้เก็บวันที่ 10 ตุลาคม 2569

6.9 แกลเลอรีและรายละเอียดการ์ด



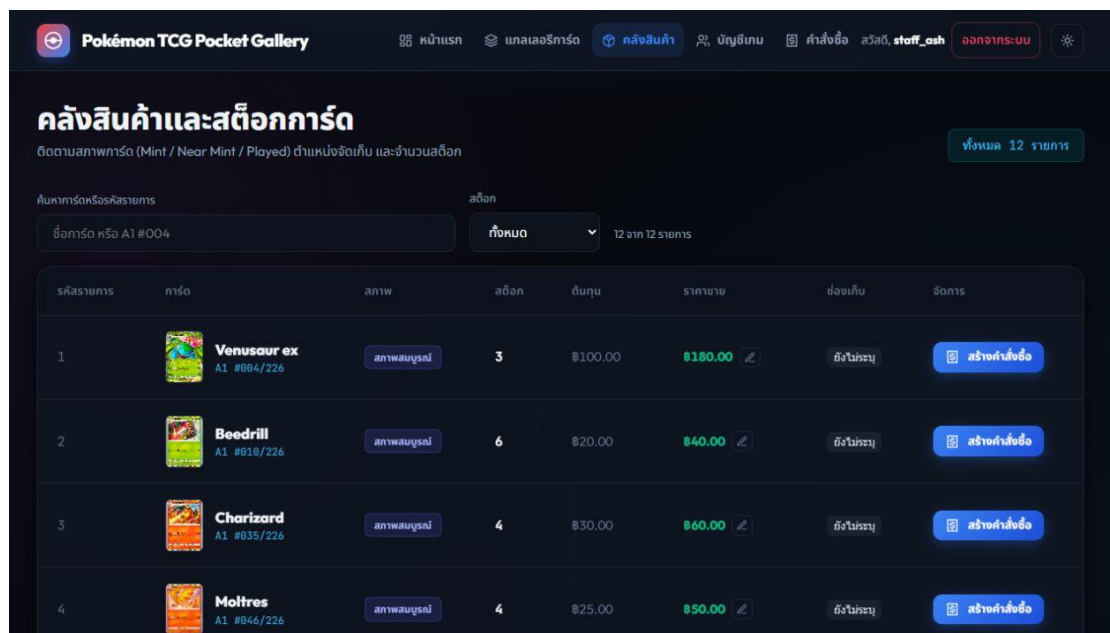
ภาพที่ 21 หน้าแกลเลอรีพร้อมการค้นหาและตัวกรอง



ภาพที่ 22 หน้าต่างรายละเอียดการ์ดและจำนวนคงเหลือ

แกลเลอรีใช้ภาพการ์ดเป็นจุดเริ่มต้นของการค้นหา รายละเอียดเปิดใน Dialog เพื่อคงบริบทของรายการเดิม ภาพและข้อมูลชื่อ ชุด เลข ราคา และสต็อกช่วยให้ตรวจสอบว่าจองการ์ดตรงไป โดยเอฟเฟกต์พลิกและไฮไลแกรมเป็นส่วนเสริมของการแสดงผล

6.10 คลังสินค้าและพรีวิวก่อนจอง

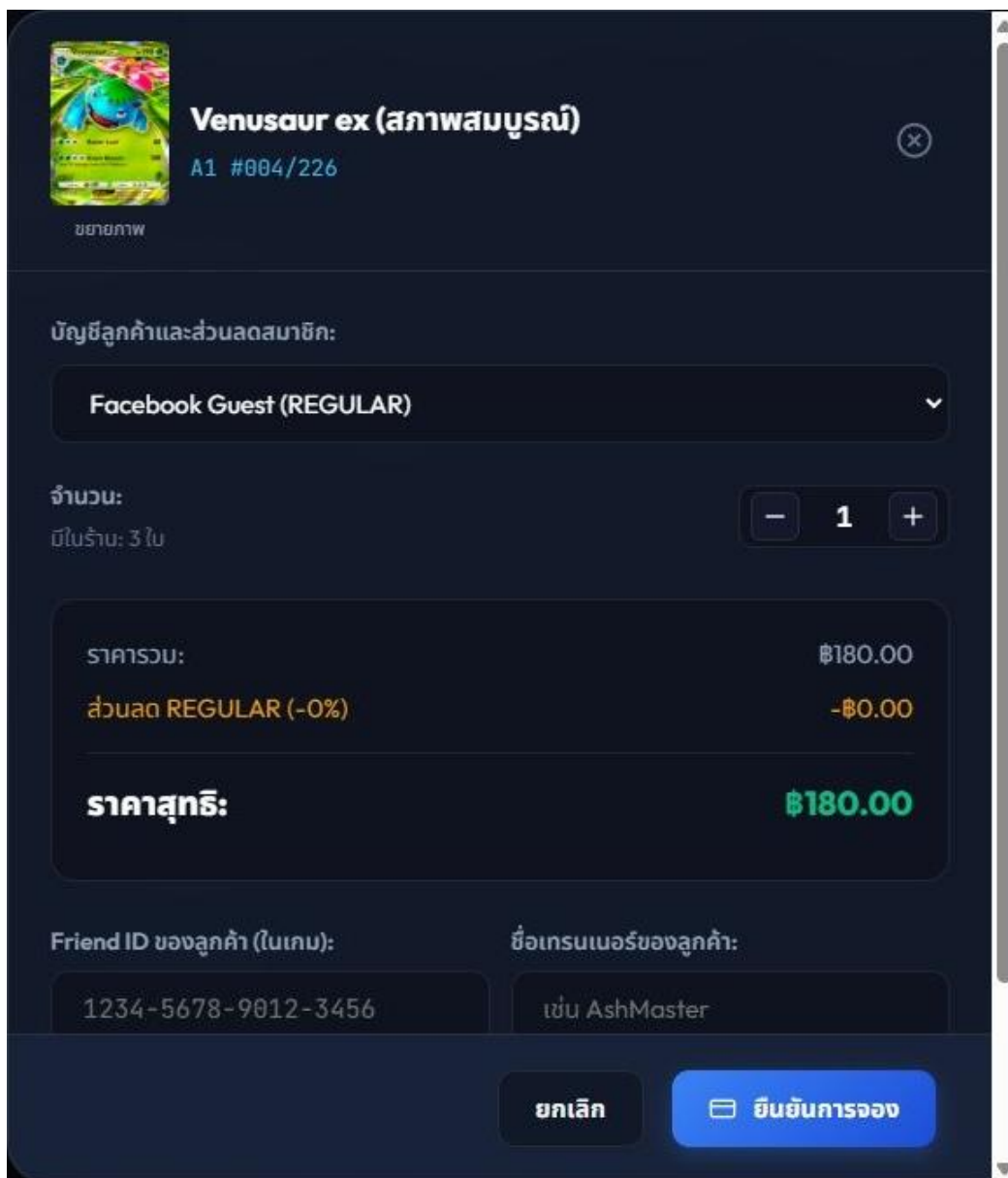


ภาพที่ 23 หน้าคลังสินค้าแสดงการ์ด สภาพ จำนวน และราคา

หน้าคลังใช้แถว Inventory เป็นหน่วยทำรายการ โดยแสดง Card สภาพ จำนวน ราคาทุน ราคาขาย และช่องเก็บ ปุ่มสร้างคำสั่งซื้ออ้างอิง Inventory ของแถวนั้น จึงไม่สับสนระหว่างการ์ดชื่อเดียวกันที่อยู่คนละสภาพหรือคนละบัญชี

การค้นหาและกรองสต็อกช่วยให้พนักงานหาการ์ดที่รับจองได้และเห็นรายการใกล้เคียง ตัวเลขสต็อกบน UI ใช้ประกอบการเลือก แต่การหักสต็อกและตรวจจำนวนเกิดซ้ำที่ Server ทุกครั้งที่ยืนยันการจอง

ประวัติการกดก่อนยืนยันการจอง



The screenshot shows a dark-themed dialog box for purchasing a Pokémon card. At the top left is the card image of Venusaur ex. To its right, the text reads "Venusaur ex (สภาพสมบูรณ์)" and "A1 #004/226". Below the card image is the label "ขยายภาพ".

Under the heading "บัญชีลูกค้าและส่วนลดสมาชิก:", there is a dropdown menu currently showing "Facebook Guest (REGULAR)".

Below this, the quantity section shows "จำนวน:" with a value of "1" and a note "มีในร้าน: 3 ใบ". To the right of the quantity are minus, plus, and reset buttons.

The price section is enclosed in a rounded rectangle. It displays:

- ราคารวม: ฿180.00
- ส่วนลด REGULAR (-0%): -฿0.00
- ราคาสุทธิ: ฿180.00 (in green)

At the bottom, there are two input fields:

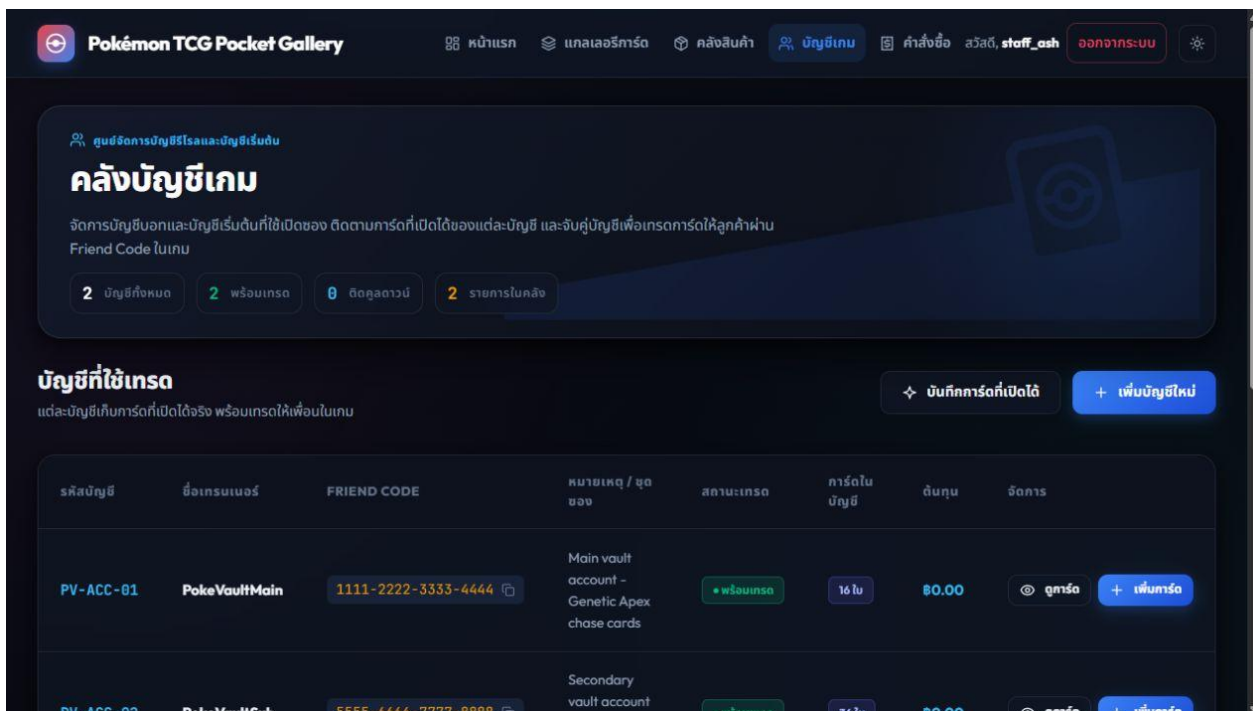
- "Friend ID ของลูกค้า (ในเกม):" with the value "1234-5678-9012-3456".
- "ชื่อเกรนเนอร์ของลูกค้า:" with the value "เช่น AshMaster".

At the very bottom are two buttons: "ยกเลิก" (Cancel) and "ยืนยันการจอง" (Confirm Reservation).

ภาพที่ 24 ประวัติการกดในหน้าต่างสร้างคำสั่งซื้อ

การเพิ่มประวัติรูปใน Dialog ช่วยตรวจสอบชื่อ เลข ชุด และสภาพที่จองได้อีกครั้งก่อนยืนยัน หน้าต่างแสดงจำนวน ราคา ส่วนลด และยอดสุทธิ พร้อมช่อง Friend ID และชื่อในเกม จึงลดการสลับกลับไปตรวจสอบ Inventory ระหว่างกรอกข้อมูล

6.11 บัญชีเกมและการเพิ่มสต็อก



ภาพที่ 25 หน้าจัดการบัญชีเกมและความพร้อมเทรด

พนักงานดูข้อมูลบัญชีเกม Friend ID สถานะเทรด และการ์ดที่อยู่ในบัญชี จากหน้านี้บันทึกการ์ดที่เปิดซองได้เข้าสู่ Inventory และเปลี่ยนสถานะความพร้อมของบัญชี ส่วนการจัดระดับสมาชิกเป็นข้อมูลผู้ใช้เว็บไซต์ที่แยกจากบัญชีเกม

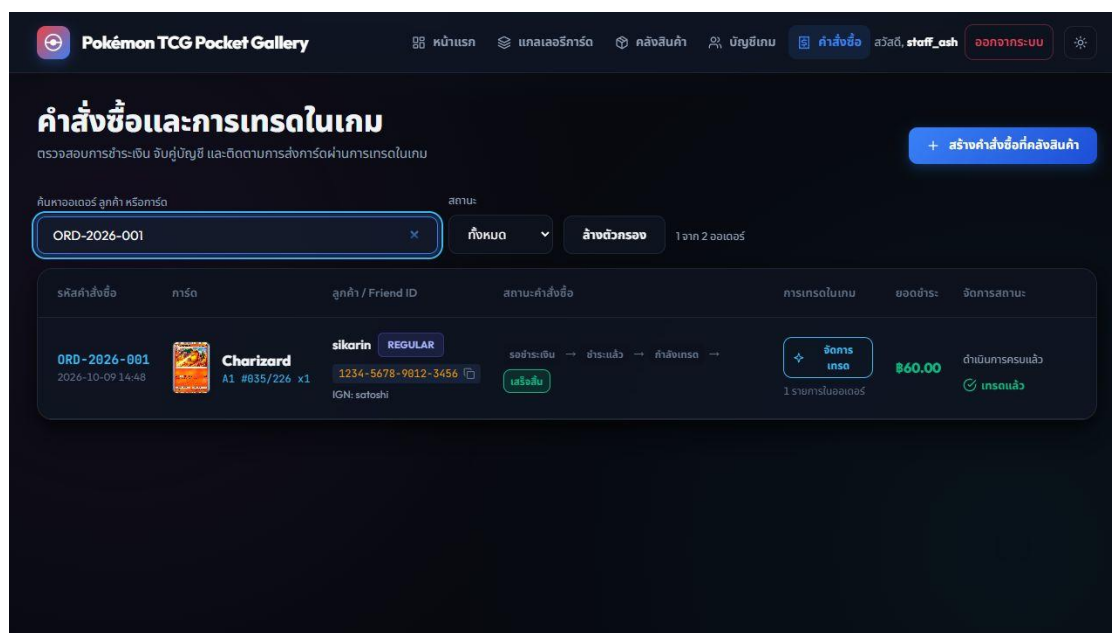
ความสัมพันธ์กับ Backend

หน้าบัญชีเรียก GameAccount API สำหรับข้อมูลบัญชีและ Add Pull Service ใช้ Card Account และ Condition คำนวณ Inventory จึงรักษาความแตกต่างของสต็อกตามบัญชี หน้าที่ยก UI คือให้เลือกข้อมูลครบและแสดงผลตอบกลับ ไม่คำนวณความพร้อมเทรดจาก Client โดยลำพัง

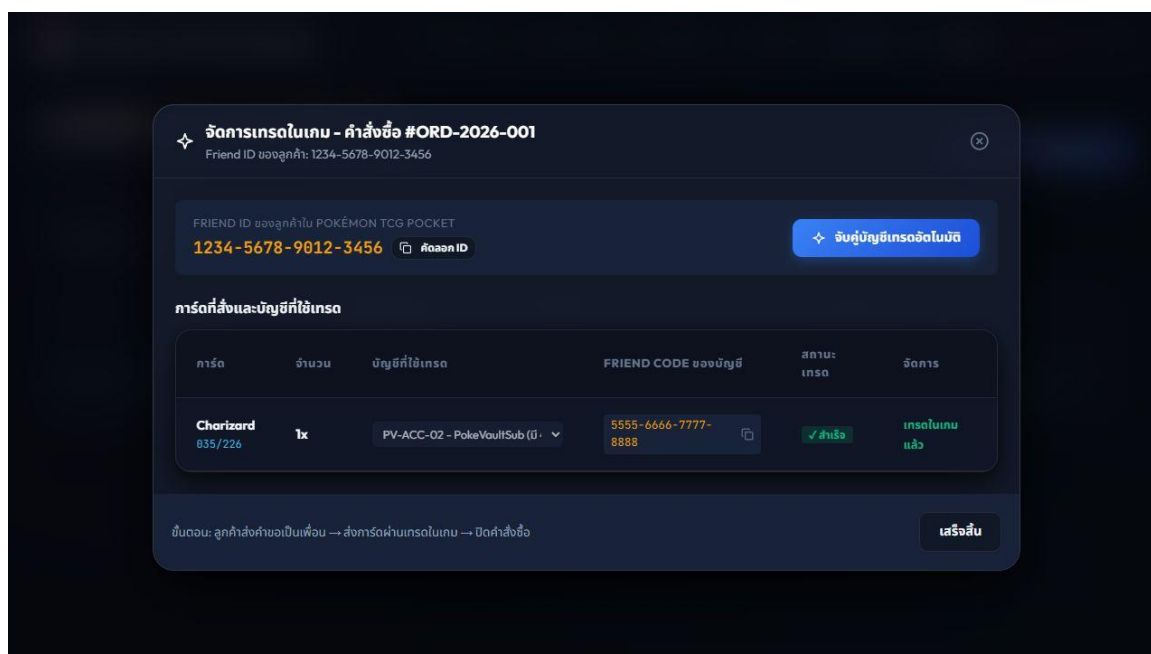
การจัดการข้อมูลอ้างอิง

บัญชีที่ยังถูก Inventory หรือ Order อ้างอิงจะไม่ถูกลบผ่าน Service เพื่อรักษาการติดตามคำสั่งซื้อ การตอบกลับข้อผิดพลาดควรบอกพนักงานว่าต้องจัดการ References ก่อน แทนการแสดงเพียงข้อความลบไม่สำเร็จ

6.12 คำสั่งซื้อและหน้าจัดการเทรด



ภาพที่ 26 หน้าคำสั่งซื้อและสถานะปัจจุบัน



ภาพที่ 27 หน้าต่างจัดการเทรดและบัญชีที่ใช้ส่งมอบ

รายการ Order แสดงสถานะและ Action ที่เกี่ยวข้อง ส่วน Trade Manager แสดง Friend ID ของลูกค้า บัญชีที่ Assign และสถานะราย Item การบันทึกการส่งมอบต้องเป็นลำดับจาก FRIEND_PENDING ไป TRADE_SENT แล้ว COMPLETED เมื่อครบทุก Item ระบบปิด Order และ UI แสดงข้อมูลสถานะล่าสุด

6.13 การจัดการภาพการ์ดใน Deployment

รูป A1a เก็บเป็น WebP ใน img/web/cards เพื่อให้ Application ใช้ไฟล์ของโครงการโดยไม่ต้องร้องขอเว็บรูปภายนอกทุกครั้ง Maven คัดลอก Resource นี้เข้า static/images ใน JAR จึงเรียกผ่าน /images/cards/... ได้ทั้ง Local และ Deployment

ตารางที่ 20 การเตรียมไฟล์รูปสำหรับแอป

ข้อมูล	รายละเอียด
จำนวนไฟล์ภาพ A1a	86 ไฟล์ WebP
ขนาดก่อนและหลังแปลง	110.96 MB เป็น 10.06 MB ตาม doc/a1a-images.md
การลดขนาด	ประมาณ 90.93 เปอร์เซ็นต์
รายการ Seed ที่เชื่อมรูป	Mew ex Celebi ex Gyarados ex และ Aerodactyl ex รวม 4 รายการ
แหล่งอ้างอิงภาพ	Limitless Pokémon TCG Pocket Card Database
Path ข้อมูลเดิม	cards.image_url

มีรูป 86 ไฟล์ไม่ได้หมายความว่า มี Catalog A1a ครบ 86 รายการในฐานข้อมูล รายงานแยกจำนวน Assets ออกจากจำนวน Seed Records เพื่อให้เข้าใจว่าโหลดครบแล้วระบบเพิ่มข้อมูลการ์ดทั้งหมดอัตโนมัติ รูปภาพเป็นทรัพย์สินของผู้ถือลิขสิทธิ์ ใช้เพื่อประกอบโครงการการศึกษาและไม่อ้างความเป็นเจ้าของ

Docker Build Context ใช้ Root Repository เพื่อให้ COPY code/src และ img/web ได้ จากนั้น Runtime Image คัดลอกเฉพาะ JAR การ Deploy ใหม่จึงยังมีภาพติดอยู่ใน Artifact ไม่พึ่งไฟล์ที่สร้างภายหลังบน Filesystem ชั่วคราวของบริการ

บทที่ 7 การทดสอบและการส่งมอบ

7.1 วิธีและขอบเขตการทดสอบ

ใช้ JUnit 5 เป็น Test Framework Mockito สำหรับแทน Dependencies และ Spring Boot Test / MockMvc สำหรับการทำงานร่วมกับ Framework มี Persistence Test และ Schema Test เพื่อเปรียบเทียบข้อกำหนดข้อมูลกับการบันทึกจริงบน H2 การแบ่งระดับช่วยแยกปัญหาที่ถูกรักออกจาก HTTP Security และ ORM

ตารางที่ 21 ระดับการทดสอบที่มีในโครงการ

ระดับ	เครื่องมือหรือ Annotation	เรื่องที่ตรวจ
Unit	JUnit 5 / Mockito	สูตรส่วนลด State Guard Matching และ Stock Rules
Controller และ Security	MockMvc / @SpringBootTest / @WithMockUser	Method Status DTO Validation และสิทธิ์
Persistence	JPA และ H2	Mapping ความสัมพันธ์ และ Rollback
Schema	schema.sql บน H2 PostgreSQL mode	Unique Check และ Foreign Key ตาม DDL
Web Models	Mock Services และ View Tests	Model Template การเข้าถึงหน้าเว็บ

7.2 ผลการรันทดสอบที่บันทึก

ตารางที่ 22 หลักฐานผลทดสอบรวม

รายการ	ผล
วันที่บันทึก	9 ตุลาคม 2569 เวลา 20:32:29 น. Asia/Bangkok
คำสั่ง	code/mvnw.cmd -f code/pom.xml clean verify
Commit	82c449b857d32dc7ad231e97a57ba3ebd976c761
Environment	Windows 11 / Java 21.0.10 / H2
Tests	373
Failures / Errors / Skipped	0 / 0 / 0
ผล Build และเวลารวม	BUILD SUCCESS / 46.613 วินาที

ผลนี้เป็น Snapshot ของ Commit และสภาพแวดล้อมที่ระบุ ไม่ใช่ Coverage 100 เปอร์เซนต์ และไม่ครอบคลุม Browser UI PostgreSQL จริง Manual Migration หรือ Cloud End to End รายงานรายสัปดาห์มีขอบเขตซ้อนกัน จึงไม่บอกยอดรายสัปดาห์เป็นยอดรวมใหม่

7.3 ตัวอย่าง Security Test ของคิธรินทร์

```

41 @Autowired MockMvc mvc;
42 @Autowired UserRepository users;
43 @Autowired PasswordEncoder encoder;
44 @MockitoBean OrderService orders;
45 @MockitoBean CardService cards;
...
110 @Test void customerCannotSpoofAnotherCustomerId() throws Exception {
111     mvc.perform(post("/api/v1/orders").with(user(red.getUsername()).roles("CUSTOMER"))
112         .contentType(MediaType.APPLICATION_JSON).content(booking(blue.getId())))
113         .andExpect(status().isForbidden());
114     verifyNoInteractions(orders);

```

Test ใช้ Spring Boot Test กับ Security Filter Chain จริง และ Mock Service ด้วย @MockitoBean ผู้ใช้จำลองเป็น CUSTOMER ส่งการจองโดยใช้ userId ของคนอื่น ต้องได้รับ 403 และ verifyNoInteractions(orders) ยืนยันว่า Service สร้างคำสั่งซื้อไม่ถูกเรียก ไม่ใช่เพียง UI ซ่อนปุ่มแล้วถือว่าปลอดภัย

ตัวอย่างกรณีที่ป้องกันความผิดพลาดสำคัญ

ตารางที่ 23 กรณีทดสอบที่สัมพันธ์กับกฎธุรกิจ

กลุ่ม	สถานการณ์	ผลที่คาด
Strategy	VIP และ WHOLESale คิดส่วนลด / subtotal ไม่ถูกต้อง	ได้ค่าตาม Contract และการปิดเศษ
State	complete ทั้งที่ Item ไม่ครบ / cancel SHIPPING	ปฏิเสธด้วย Domain Exception
Stock	หลาย Item แต่รายการหลังสต็อกไม่พอ	ย้อนกลับการจองทั้ง Transaction
Matching	จองใบสุดท้ายแล้วเลือก Inventory เดิม	นับ Reservation ใน Effective Stock
Ownership	ลูกค้าอ่านหรือสร้างแทนเจ้าของอื่น	403 และไม่เข้าถึงข้อมูล
Observer	สต็อกรวมเหลือไม่เกิน 2 ใบ	ตรวจ Threshold และพฤติกรรม Listener

สรุปผลทุก Suite อยู่ในภาคผนวก ข และ XML ใน test/reports/project ตรวจสอบได้ รายงานนี้ไม่ได้เพิ่มผลทดสอบใหม่จากการจัดทำเอกสาร

7.4 ทีมและ Git Workflow

สมาชิกทำงานบน Branch ของตน เปิด Pull Request ไปยัง develop ให้เพื่อนตรวจสอบความเชื่อมโยงกับ Module ถัดไป แล้วรวม Version ส่งมอบเข้า main การแบ่งงานตามโดเมนช่วยให้ทุกคนอธิบายโค้ดของตนและเข้าใจจุดเชื่อมกับทีม

ตารางที่ 24 สมาชิกและความรับผิดชอบ

รหัสและชื่อ	ขอบเขตงาน	Branch
673380292-5 นายศิรินทร์ อุปจันทร์	Catalog และ Security	sikarin_6733802925_01
673380272-1 นายธนภูมิ จันทรา	Order และ Strategy	tanapoom_6733802721_01
673380294-1 นายสรวิชัย ศาสสนสุพันธุ์	Frontend และ DevOps	soravit_6733802941_02
673380066-4 นายสัพพัญญู คำคุ้ม	Vault และ Observer	Sapphanyukhamtoom_6733800664_01
673380301-0 นายแทนคุณ พันธนิกุล	State และ Trade Matching	Tankun_6733803010_01

จุดที่ใช้ตรวจร่วมกัน

- Catalog → Vault ใช้ Entity Card และข้อมูลสภาพการård
- Vault → Order ใช้ Inventory และกฎการหัก Stock
- Order → Trade ใช้ Order State และ Reservation
- Trade → Frontend ใช้สถานะราย Item และ Recommendation
- Frontend → Core ตรวจ Security DTO และ Deployment Configuration

การประเมินรายบุคคลควรตรวจ Git Log Contributors Pull Requests และ Review ของ Branch จริง เอกสารแบ่งหน้าที่ไม่ใช่หลักฐานแทนจำนวนหรือคุณภาพ Commit จึงไม่สร้างตารางยืนยันยอด Commit หรือ Approval ที่ไม่ได้ตรวจในประวัติจริง

7.5 โครงสร้าง Repository และ CI

```
Pok-Vault_Commerce/
code/
  pom.xml mvnw mvnw.cmd Dockerfile
src/main/java/com/pokevault/
  common/ domain/ repository/ modules/
src/main/resources/
  templates/ static/ db/manual/
  application.yml schema.sql data.sql
test/
  java/ resources/ reports/
doc/
  diagrams/uml/ slide/ report/
  solid-analysis.md design-patterns.md
img/
  screenshots/ uml/ web/cards/
.github/workflows/ci-cd.yml
README.md
```

Production Code อยู่ code/src/main Tests อยู่ test/java โดย pom.xml กำหนด testSourceDirectory ให้ Maven ค้นตามโครงสร้างส่งงาน รูปที่แอปใช้แยกใน img/web ส่วน img/uml และ img/screenshots เป็นหลักฐานเอกสาร README มีสมาชิก วิธีรัน API และ URL Deployment

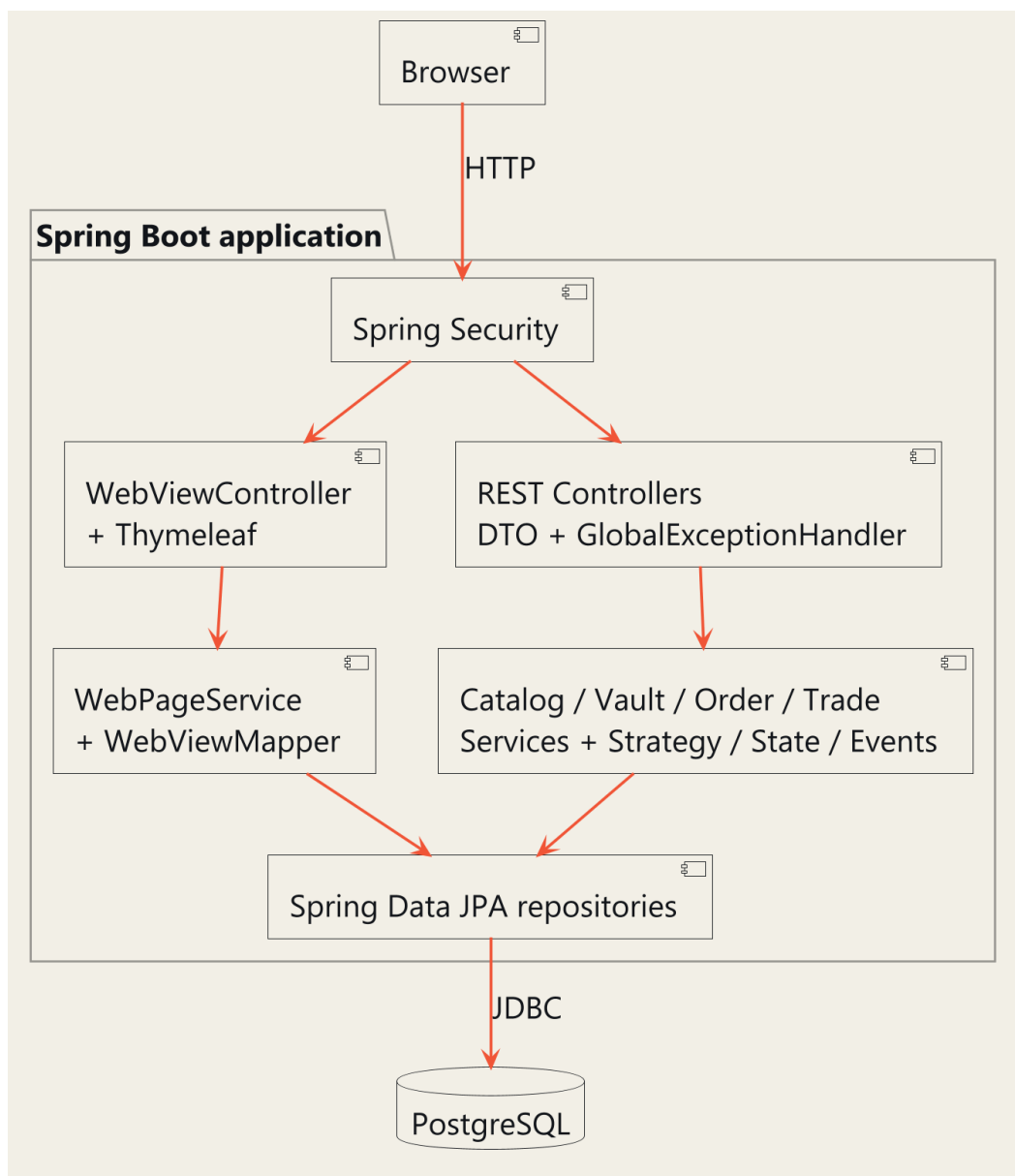
GitHub Actions

Workflow ทำงานเมื่อ Pull Request หรือ Push ไป develop และ main Job test ใช้ JDK 17 และคำสั่ง clean verify ตั้งค่า PostgreSQL Service และอัปโหลด Surefire Reports แม้ Job ล้มเหลว Job build-and-push รอ Test และทำเฉพาะ Push เพื่อสร้าง Docker Image แล้วส่ง GHCR

การตั้ง Environment เป็น prod ใน Workflow ไม่ได้แปลว่า Test ทุก Suite เชื่อม PostgreSQL เพราะบาง Suite ระบุ H2 ของตนเอง ต้องอ่าน Log และผล CI รอบนั้นประกอบ อีกทั้ง Workflow ปัจจุบันไม่มี Job Deploy ไป Render จึงไม่อ้างว่า Pipeline นี้เป็น Build Test Deploy อัตโนมัติครบวงจร

7.6 Component Diagram

Component Diagram แสดงหน่วยซอฟต์แวร์ของ Application ได้แก่ Browser Views REST และ Web Controllers Services Repositories และส่วน Framework Config Security และ Database การแยก Module เป็นการจัดโครงสร้างใน Application เดียว ไม่ใช่บริการเครือข่ายคนละชุด

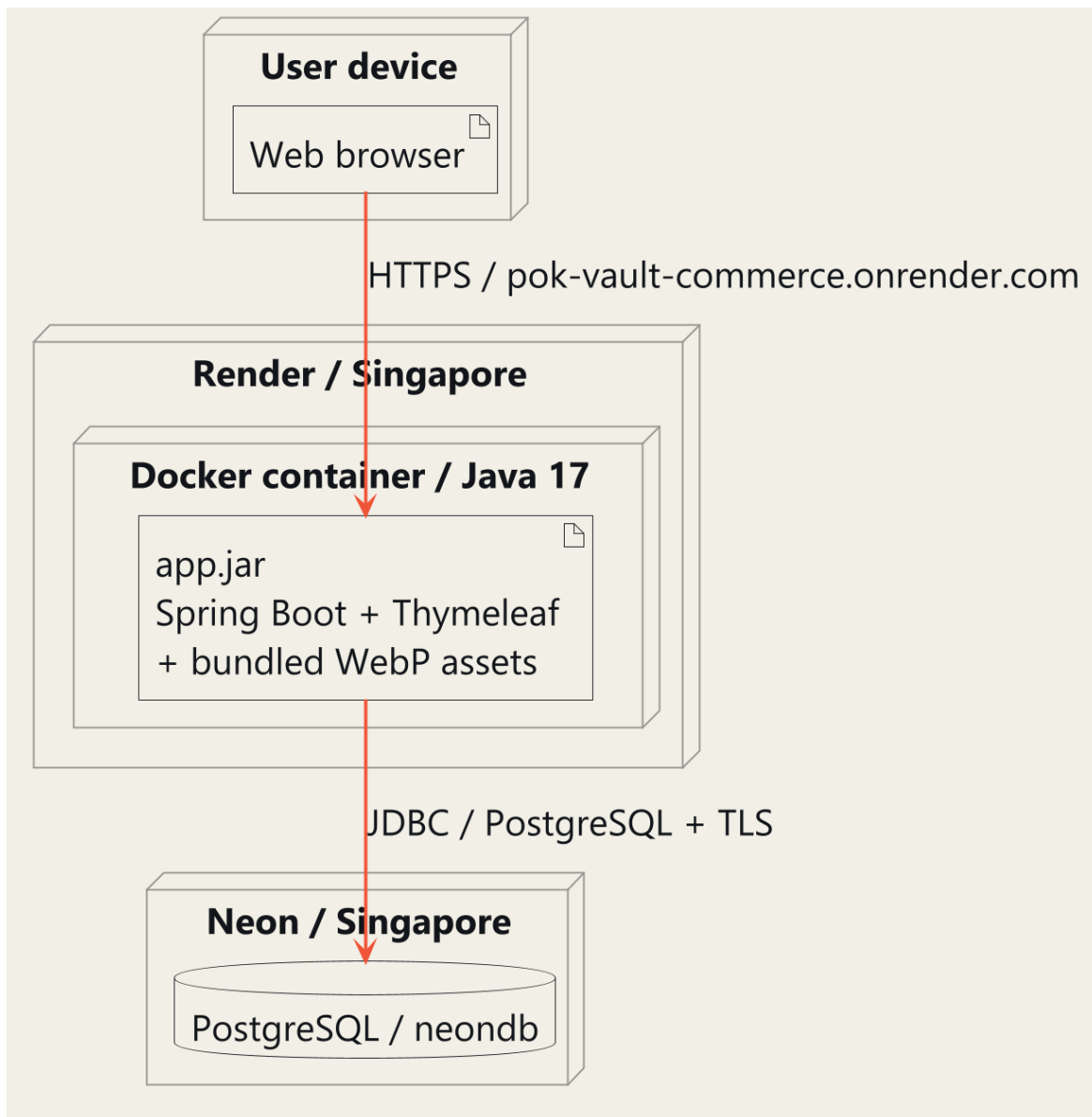


ภาพที่ 28 Component Diagram ของระบบ

การเชื่อม Browser กับ Backend มีทั้งการรับหน้า Thymeleaf และการเรียก REST ส่วน Services ใช้ DTO และ Entity ตามขอบเขตงาน Spring Security และ Exception Handling เป็นองค์ประกอบที่ประมวลผลก่อนหรือรอบ Controller

7.7 Deployment Diagram

Application บรรจุใน Docker และทำงานเป็น Web Service บน Render ผู้ใช้เข้า HTTPS ผ่าน Browser ฐานข้อมูล PostgreSQL อยู่ที่ Neon โดย Application เชื่อมผ่าน JDBC และ TLS ความคงทนของข้อมูลจึงอยู่ที่บริการฐานข้อมูล ส่วนรูป WebP เป็น Resource ใน JAR



ภาพที่ 29 Deployment Diagram บน Render และ Neon

Diagram แยก Node Execution Environment และ Artifact app.jar ออกจาก Components ทางตรรกะ ได้ดบน GitHub และ Image ที่ GHCR เป็น Artifact ฝั่ง Build ส่วน Render และ Neon เป็น Runtime การมี Image ที่ Build ผ่านไม่ใช่หลักฐานว่า Public URL พร้อมใช้งานตลอดเวลา

7.8 การติดตั้งและค่าที่ต้องเตรียม

Local Development

```
code\mvnw.cmd -f code\pom.xml spring-boot:run
```

ใช้ Java 17 ขึ้นไปและ Maven Wrapper Profile Default คือ local ใช้ H2 เปิดเว็บที่ <http://localhost:8080> หรือ Port ที่กำหนดเอง บัญชี Seed สำหรับสาธิตคือ admin staff_ash และ customer_red ซึ่งมีบทบาท ADMIN STAFF และ CUSTOMER ระดับ VIP ตามลำดับ รหัสผ่านสาธิต password123

Render และ Neon

ตารางที่ 25 Configuration สำหรับ Deployment

ค่าหรือช่อง	ค่าที่ใช้หรือรูปแบบ
Runtime / Branch	Docker / main
Dockerfile Path	code/Dockerfile
Docker Build Context	Root Repository หรือ .
SPRING_PROFILES_ACTIVE	prod
SPRING_DATASOURCE_URL	jdbc:postgresql://<host>:5432/neondb?sslmode=require
SPRING_DATASOURCE_USERNAME	Role ของ PostgreSQL ที่เลือกใน Neon
SPRING_DATASOURCE_PASSWORD	เก็บใน Secret Environment Variable
PORT	ค่าที่ Host กำหนดให้ Server
Health Check Path	/actuator/health

Dockerfile มี Build Stage จาก Maven และ JDK 17 จากนั้น Runtime Stage ใช้ JRE 17 และคัดลอกเฉพาะ JAR Build ใช้ -DskipTests จึงต้องดูผล CI หรือ Test Report แยก ไม่ถือว่า Docker Build ได้รัน Tests แล้ว Connection String ในรายงานเป็นรูปแบบตัวอย่าง ไม่รวม Host Credential หรือ Password จริง

บริการแบบ Free อาจใช้เวลาเริ่มทำงานหลังไม่มีการใช้งาน ก่อนนำเสนอควรเปิดหน้าเว็บและ Swagger ให้พร้อม และเตรียม Seed Data กับภาพหรือวิดีโอสำรอง ภาพ Production ในรายงานเก็บได้หลังบริการเริ่มทำงานแล้ว ไม่ใช่การรับรอง Uptime หรือ Load Capacity

บทที่ 8 ผลงาน ข้อจำกัด และข้อสรุป

8.1 ผลงานที่ได้

ระบบเชื่อมงานเลือกการ์ด การจอง สต็อก และการเทรดได้ใน Application เดียว มี REST CRUD สอง Resources พร้อมหน้าร้านและหน้าจัดการ การใช้ Strategy State และ Observer แสดงการแยกกฎที่เปลี่ยนตามข้อมูลและเหตุการณ์ได้ตรงกับปัญหา Diagram และโค้ดอ้างอิงชื่อคลาส ความสัมพันธ์ และลำดับ Use Case ชัดเดียวกัน

8.2 บทเรียนจากการพัฒนา

ความผิดพลาดที่พบในกระบวนการทำงานคือการสื่อสารสถานะ Order กับ Item ไม่ชัดเจน ผู้ใช้พยายาม complete Order ก่อนส่งมอบครบ หรือกดซ้ำหลัง Item สุดท้ายปิด Order อัตโนมัติ บทเรียนคือ UI ต้องสื่อสารสถานะล่าสุดจาก Server และ Backend ต้องมี Guard รวมถึงพฤติกรรม Idempotent ในจุดที่เหมาะสม

การจับคู่บัญชีต้องคิดสต็อกที่ถูกจองแล้วด้วย หากดูเฉพาะ quantity ที่เหลืออาจปฏิเสธคำสั่งซื้อที่จองใบสุดท้ายไปแล้ว กลุ่มจึงใช้ Effective Stock สำหรับ Inventory ที่ถือ Reservation และย้าย Reservation เมื่อเปลี่ยนบัญชี ความสัมพันธ์ของข้อมูลจึงต้องอธิบายคู่กับ Transaction

8.3 ข้อจำกัดและแนวทางพัฒนาต่อ

- ปรับ Migration เป็น Versioned Migration และเพิ่มการทดสอบบน PostgreSQL จริง
- เพิ่ม Browser End to End และ Concurrent Booking Tests ก่อนใช้งานปริมาณมาก
- ทบทวน CSRF และการเผยแพร่ข้อมูล Error ในสภาพแวดล้อม Production
- เพิ่มดัชนีจาก Query Plan และพิจารณา Lock หรือ Versioning ของ Stock
- ปรับ Event สำหรับการแจ้งเตือนภายนอกให้ทำงานหลัง Commit
- เชื่อม CI กับ Deployment พร้อมหลักฐานผลรัน หากต้องการ Pipeline ครบวงจร

8.4 ข้อสรุป

โครงการแสดงการประยุกต์หลักการออกแบบซอฟต์แวร์ผ่านโค้ดที่ทำงานจริงและหลักฐานตรวจสอบได้ ผลทดสอบที่มีสนับสนุนกฎธุรกิจและการเข้าถึงข้อมูลตามขอบเขตของ Suite แต่การใช้งานนอกบริบทการศึกษายังต้องเพิ่มการทดสอบและควบคุมการดำเนินงานตามข้อจำกัดที่ระบุ

เอกสารอ้างอิง

[1] Spring Spring Boot Reference 3.4

<https://docs.spring.io/spring-boot/3.4/reference/>

[2] PlantUML Class Diagram และคู่มือ UML Diagrams

<https://plantuml.com/class-diagram>

[3] JUnit Team JUnit 5 User Guide 5.11.4

<https://docs.junit.org/5.11.4/user-guide/>

[4] OpenAPI Initiative OpenAPI Specification 3.1.0

<https://spec.openapis.org/oas/v3.1.0.html>

[5] PostgreSQL Global Development Group PostgreSQL 16 Constraints

<https://www.postgresql.org/docs/16/ddl-constraints.html>

[6] Render Docker on Render

<https://render.com/docs/docker>

[7] Neon Connect from Java

<https://neon.com/docs/guides/java>

[8] Spring Spring Data JPA Reference

<https://docs.spring.io/spring-data/jpa/reference/>

[9] Limitless Pokémon TCG Pocket Mythical Island A1a

<https://pocket.limitlesstcg.com/cards/A1a>

สืบค้นวันที่ 10 ตุลาคม 2569 คู่มือ Spring Boot ใน URL อ้างอิงเป็นชุดเอกสารสาขา 3.4 ส่วนเวอร์ชัน Application ที่ระบุใน pom.xml คือ 3.4.3

หลักฐานภายในโครงการ

README.md ขอบเขต สมาชิก วิธีรัน API และ Deployment

doc/solid-analysis.md เหตุผล SOLID พร้อมไฟล์และบรรทัด

doc/design-patterns.md ตำแหน่งและเหตุผลการใช้ Patterns

doc/diagrams/uml/ ต้นฉบับ PlantUML และ Manifest

doc/diagrams/UML-AUDIT.md ผลตรวจ Diagram กับโค้ดและ Schema

code/src/main/resources/schema.sql DDL ต้นฉบับทั้ง 8 ตาราง

test/reports/project/TEST-REPORT.md Snapshot ผลทดสอบรวม 373 กรณี

ฐานโค้ดที่ใช้อธิบายการออกแบบคือ a55e5dfe43012e7f4df92effe9e11c7577364c4 ส่วน Test Report ใช้ Commit 82c449b857d32dc7ad231e97a57ba3ebd976c761 ตามวันที่ในรายงานทดสอบ การตรวจ UML วันที่ 10 ตุลาคม 2569 เป็นการตรวจ Source และ Schema ไม่ได้สร้างผล JUnit รอบใหม่

ภาคผนวก ก Data Dictionary

พจนานุกรมข้อมูลต่อไปนี้จะสร้างรายการคอลัมน์จาก schema.sql โดยตรง ครอบคลุมชื่อ SQL Type Nullability ค่า Default และคีย์สำคัญ PK หมายถึง Primary Key FK หมายถึง Foreign Key และ UK หมายถึง Unique Constraint เครื่องหมาย — หมายถึงไม่มีค่า Default ที่ระบุใน DDL

ชนิด BIGSERIAL เป็น คีย์ ที่ สร้าง ค่า จาก Sequence created_at เป็น NOT NULL และมี Default CURRENT_TIMESTAMP ส่วน updated_at มี Default แต่ยอมรับ NULL ตาม DDL Enum ใน Java เก็บเป็น VARCHAR ใน SQL การตรวจ Enum ที่ API ไม่ได้แปลว่ามี CHECK รายค่า Enum ในฐานข้อมูล

ตารางที่ 26 ภาพรวม Data Dictionary

ตาราง	หน้าที่	คอลัมน์
users	บัญชีผู้ใช้เว็บไซต์	7
user_profiles	โปรไฟล์และระดับสมาชิก	9
card_expansions	ชุดการ์ด	8
cards	ตัวตนของการ์ด	12
game_accounts	บัญชีเกมของร้าน	9
card_inventories	สต็อกและราคาในแต่ละบัญชี	10
orders	คำสั่งซื้อ	12
order_items	รายการและการส่งมอบ	10

ก users

ตารางที่ 27 Data Dictionary ของ users

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
username	VARCHAR(50)	NO	—	UK; ชื่อเข้าสู่ระบบ ไม่ซ้ำ
email	VARCHAR(100)	NO	—	UK; อีเมลผู้ใช้ ไม่ซ้ำ
password	VARCHAR(255)	NO	—	Password Hash ของบัญชีเว็บ
role	VARCHAR(30)	NO	—	ADMIN STAFF หรือ CUSTOMER
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

uk_user_username UNIQUE (username)

uk_user_email UNIQUE (email)

ก user_profiles

ตารางที่ 28 Data Dictionary ของ user_profiles

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
user_id	BIGINT	NO	—	FK และ UK; FK ไป users.id
full_name	VARCHAR(100)	YES	—	ชื่อผู้ใช้หรือชื่อติดต่อ
phone_number	VARCHAR(20)	YES	—	หมายเลขโทรศัพท์
shipping_address	VARCHAR(500)	YES	—	ข้อมูลที่อยู่ประกอบโปรไฟล์
membership_tier	VARCHAR(30)	NO	'REGULAR'	REGULAR VIP หรือ WHOLESALE
reward_points	INTEGER	NO	0	คะแนนสมาชิก ไม่ติดลบ
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

uk_profile_user UNIQUE (user_id)

fk_profile_user FK (user_id) → users(id) ON DELETE CASCADE

chk_profile_reward_points CHECK (reward_points >= 0)

ก card_expansions

ตารางที่ 29 Data Dictionary ของ card_expansions

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
code	VARCHAR(20)	NO	—	UK; รหัสชุด เช่น A1 หรือ A1a
name	VARCHAR(100)	NO	—	ชื่อชุดหรือชื่อการ์ด
series	VARCHAR(50)	NO	—	ชื่อ Series
release_date	DATE	YES	—	วันที่ออกชุด
total_cards	INTEGER	NO	—	จำนวนการ์ดตามข้อมูลชุด ต้องมากกว่า 0
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

uk_expansion_code UNIQUE (code)

chk_expansion_total CHECK (total_cards > 0)

ก cards

ตารางที่ 30 Data Dictionary ของ cards

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
expansion_id	BIGINT	NO	—	FK และ UK คู่; FK ไป card_expansions.id
card_number	VARCHAR(20)	NO	—	UK คู่ expansion_id; เลขการ์ดภายในชุด
name	VARCHAR(100)	NO	—	ชื่อชุดหรือชื่อการ์ด
card_type	VARCHAR(30)	NO	—	ประเภทการ์ด
rarity	VARCHAR(30)	NO	—	ระดับความหายาก
element_type	VARCHAR(30)	YES	—	ธาตุของการ์ด
hp	INTEGER	YES	—	ค่า HP ไม่ติดลบเมื่อมีค่า
retreat_cost	INTEGER	YES	—	ค่าถอย ไม่ติดลบเมื่อมีค่า
image_url	VARCHAR(500)	YES	—	Path หรือ URL รูปการ์ด
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

fk_card_expansion FK (expansion_id) → card_expansions(id) ON DELETE RESTRICT

uk_card_expansion_number UNIQUE (expansion_id, card_number)

chk_card_hp CHECK (hp >= 0)

chk_card_retreat CHECK (retreat_cost >= 0)

ก game_accounts

ตารางที่ 31 Data Dictionary ของ game_accounts

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
account_code	VARCHAR(50)	NO	—	UK; รหัสบัญชีเกม ไม่ซ้ำ
in_game_name	VARCHAR(100)	NO	—	ชื่อในเกมของบัญชีร้าน
friend_id	VARCHAR(50)	NO	—	Friend ID ในเกม
trade_status	VARCHAR(30)	NO	'READY'	READY BUSY_TRADING COOLDOWN SUSPENDED
buy_in_cost	DECIMAL(10,2)	YES	0.00	ต้นทุนของบัญชี ไม่ติดลบ
notes	VARCHAR(500)	YES	—	หมายเหตุของรายการ
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

uk_account_code UNIQUE (account_code)

chk_account_buy_in CHECK (buy_in_cost >= 0.00)

น card_inventories

ตารางที่ 32 Data Dictionary ของ card_inventories

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
card_id	BIGINT	NO	—	FK; FK ไป cards.id
game_account_id	BIGINT	YES	—	FK; FK ไป game_accounts.id เลือกเป็น NULL ได้
card_condition	VARCHAR(30)	NO	—	สภาพการ์ด
quantity	INTEGER	NO	0	จำนวนคงเหลือ ≥ 0
buy_in_price	DECIMAL(10,2)	NO	0.00	ราคาทุนต่อใบ ไม่ติดลบ
selling_price	DECIMAL(10,2)	NO	0.00	ราคาขายต่อใบ ไม่ติดลบ
storage_slot	VARCHAR(50)	YES	—	ช่องหรือชื่อที่จัดเก็บ
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

fk_inventory_card FK (card_id) → cards(id) ON DELETE RESTRICT

fk_inventory_game_account FK (game_account_id) → game_accounts(id) ON DELETE SET NULL

chk_inventory_qty CHECK (quantity ≥ 0)

chk_inventory_buy_price CHECK (buy_in_price ≥ 0.00)

chk_inventory_sell_price CHECK (selling_price ≥ 0.00)

ก orders

ตารางที่ 33 Data Dictionary ของ orders

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
order_code	VARCHAR(50)	NO	—	UK; รหัสคำสั่งซื้อ ไม่ซ้ำ
user_id	BIGINT	NO	—	FK; FK ไป users.id
customer_friend_id	VARCHAR(50)	YES	—	Friend ID ลูกค้าสำหรับส่งมอบ
customer_in_game_name	VARCHAR(100)	YES	—	ชื่อในเกมของลูกค้า
order_status	VARCHAR(30)	NO	'PENDING'	PENDING PAID SHIPPING COMPLETED CANCELLED
total_amount	DECIMAL(10,2)	NO	0.00	ยอดรวมก่อนส่วนลด
discount_amount	DECIMAL(10,2)	NO	0.00	จำนวนเงินส่วนลด
final_amount	DECIMAL(10,2)	NO	0.00	ยอดสุทธิ
notes	VARCHAR(500)	YES	—	หมายเหตุของรายการ
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

uk_order_code UNIQUE (order_code)

fk_order_user FK (user_id) → users(id) ON DELETE RESTRICT

chk_order_total CHECK (total_amount >= 0.00)

chk_order_discount CHECK (discount_amount >= 0.00)

chk_order_final CHECK (final_amount >= 0.00)

ก order_items

ตารางที่ 34 Data Dictionary ของ order_items

Column	SQL Type	NULL	Default	คีย์และความหมาย
id	BIGSERIAL	NO	—	PK; รหัสหลักของแถว
order_id	BIGINT	NO	—	FK; FK ไป orders.id
inventory_id	BIGINT	NO	—	FK; FK ไป card_inventories.id
assigned_account_id	BIGINT	YES	—	FK; FK ไป game_accounts.id เลือกเป็น NULL ได้
trade_status	VARCHAR(30)	YES	'UNASSIGNED'	UNASSIGNED FRIEND_PENDING TRADE_SENT COMPLETED
quantity	INTEGER	NO	1	จำนวนที่สั่ง ≥ 1
unit_price	DECIMAL(10,2)	NO	0.00	ราคาต่อใบ ณ เวลาจอง
subtotal	DECIMAL(10,2)	NO	0.00	ราคารวมของ Item ณ เวลาจอง
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	เวลาสร้างข้อมูล
updated_at	TIMESTAMP	YES	CURRENT_TIMESTAMP	เวลาแก้ไขล่าสุด

Constraints ของตาราง

fk_item_order FK (order_id) → orders(id) ON DELETE CASCADE

fk_item_inventory FK (inventory_id) → card_inventories(id) ON DELETE RESTRICT

fk_item_assigned_account FK (assigned_account_id) → game_accounts(id) ON DELETE SET NULL

chk_item_quantity CHECK (quantity ≥ 1)

chk_item_unit_price CHECK (unit_price ≥ 0.00)

chk_item_subtotal CHECK (subtotal ≥ 0.00)

ภาคผนวก ข ผลทดสอบราย Suite

ตารางต่อไปนี้มาจากผล Surefire รอบเดียวกับ Test Report รวม แถวที่มี Tests เท่ากับ 0 อาจเป็น Parent Suite ที่มีผลจริงใน Nested Suites จึงแสดงตามไฟล์สรุปเดิม และรวมผลจำนวนกรณีจากทุกแถวเพียงครั้งเดียว

ตารางที่ 35 ผลทดสอบราย Suite ชุด 1

Suite	Tests	Fail	Error	Skip
common.exception.CustomExceptionTest	2	0	0	0
common.security.ApiAuthorizationTest	36	0	0	0
common.security.CustomUserDetailsServiceTest	3	0	0	0
common.security.OrderAccessPolicyTest	3	0	0	0
domain.entity.CardInventoryTest	5	0	0	0
modules.catalog.CardApiControllerTest	19	0	0	0
modules.catalog.CardServiceEdgeCaseTest	13	0	0	0
modules.catalog.CardServiceTest	13	0	0	0
modules.catalog.CatalogPersistenceTest	7	0	0	0
modules.catalog.CatalogSchemaSqlTest	6	0	0	0
modules.catalog.UserServiceTest	8	0	0	0
modules.order.DiscountStrategyEdgeCaseTest	11	0	0	0
modules.order.DiscountStrategyTest	6	0	0	0
modules.order.OrderApiControllerTest	15	0	0	0
modules.order.OrderBookingApiControllerTest	24	0	0	0
modules.order.OrderBookingApiSecurityTest	2	0	0	0
modules.order.OrderPersistenceTest	7	0	0	0

ข ผลทดสอบราย Suite ต่อ

ตารางที่ 36 ผลทดสอบราย Suite ชุด 2

Suite	Tests	Fail	Error	Skip
modules.order.OrderServiceEdgeCaseTest	8	0	0	0
modules.order.OrderServiceTest	23	0	0	0
modules.order.OrderStockAndSecurityTest CancelOrderRestorationTests	1	0	0	0
modules.order.OrderStockAndSecurityTest LastCardMatchingTests	1	0	0	0
modules.order.OrderStockAndSecurityTest MultitemRollbackTests	1	0	0	0
modules.order.OrderStockAndSecurityTest OwnershipSecurityTests	5	0	0	0
modules.order.OrderStockAndSecurityTest ReassignAccountTests	5	0	0	0
modules.order.OrderStockAndSecurityTest	0	0	0	0
modules.trade.advice.GlobalExceptionHandlerTest	10	0	0	0
modules.trade.controller.TradeMatchingApiControllerTest	13	0	0	0
modules.trade.dto.TradeRecommendationResponseTest	3	0	0	0
modules.trade.service.TradeMatchingServiceTest AutoMatchAlgorithmTests	6	0	0	0
modules.trade.service.TradeMatchingServiceTest BatchAutoMatchTests	2	0	0	0
modules.trade.service.TradeMatchingServiceTest CardConditionMatchingTests	7	0	0	0
modules.trade.service.TradeMatchingServiceTest ManualAssignmentTests	9	0	0	0
modules.trade.service.TradeMatchingServiceTest RecommendationQueryTests	4	0	0	0
modules.trade.service.TradeMatchingServiceTest	0	0	0	0

ข ผลทดสอบราย Suite ต่อ

ตารางที่ 37 ผลทดสอบราย Suite ชุด 3

Suite	Tests	Fail	Error	Skip
modules.trade.state.OrderStateTest CancellationTests	4	0	0	0
modules.trade.state.OrderStateTest CancelledOrderStateEdgeCaseTests	3	0	0	0
modules.trade.state.OrderStateTest ExecuteActionStringTests	2	0	0	0
modules.trade.state.OrderStateTest FactoryMethodTests	2	0	0	0
modules.trade.state.OrderStateTest HappyPathTests	1	0	0	0
modules.trade.state.OrderStateTest IllegalTransitionTests	3	0	0	0
modules.trade.state.OrderStateTest OrderContextMutatorTests	3	0	0	0
modules.trade.state.OrderStateTest ShippingGuardTests	3	0	0	0
modules.trade.state.OrderStateTest TerminalStateTests	2	0	0	0
modules.trade.state.OrderStateTest	0	0	0	0
modules.vault.observer.LowStockObserverTest	7	0	0	0
modules.vault.service.GameAccountServiceTest AccountManagementTests	12	0	0	0
modules.vault.service.GameAccountServiceTest AddPulledCardTests	4	0	0	0
modules.vault.service.GameAccountServiceTest	0	0	0	0
modules.web.PlatformEndpointsTest	4	0	0	0
modules.web.RegistrationServiceTest	3	0	0	0
modules.web.StoreAdminServiceTest	5	0	0	0

ข ผลทดสอบราย Suite ต่อ

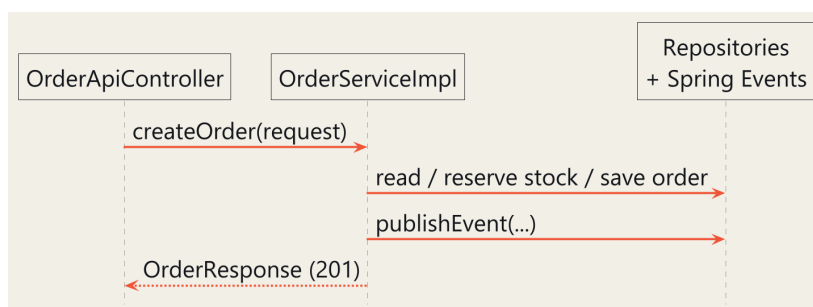
ตารางที่ 38 ผลทดสอบราย Suite ชุด 4

Suite	Tests	Fail	Error	Skip
modules.web.WebPageServiceTest CardGallery	13	0	0	0
modules.web.WebPageServiceTest	0	0	0	0
modules.web.WebViewControllerTest AccessAndNavbar	11	0	0	0
modules.web.WebViewControllerTest LoginPage	5	0	0	0
modules.web.WebViewControllerTest PageModels	8	0	0	0
modules.web.WebViewControllerTest	0	0	0	0

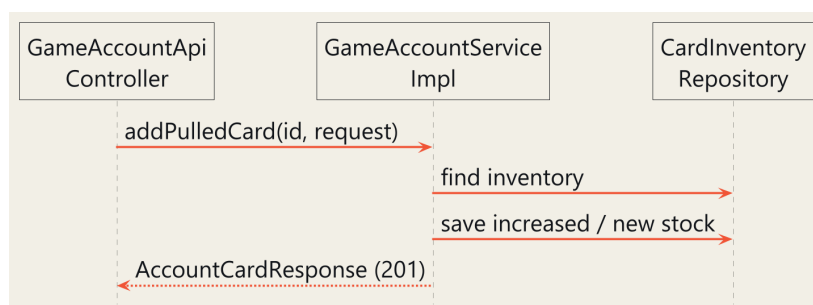
รวม 373 Tests โดย Failures Errors และ Skipped เท่ากับ 0 ขอบเขตผลรันเป็น H2 ตามบทที่ 7 ไม่รวมผล Browser หรือผลทดสอบ Neon

ภาคผนวก ค Diagram ฉบับสรุป

ภาพฉบับสรุปต่อไปนี้เป็นต้นฉบับ PlantUML ชุดเดียวกับสไลด์ ใช้ย่อขั้นตอนหรือเลือกเฉพาะโครงสร้างที่ต้องอธิบายบนหน้าจอ ส่วน Diagram รายละเอียดในบทหลักยังเป็นหลักฐานของลำดับจริงและเงื่อนไขครบถ้วน

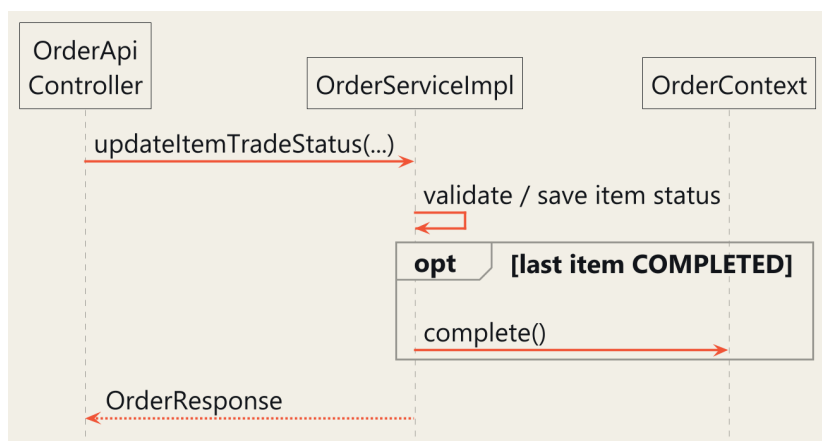


ภาพที่ 30 Sequence ฉบับสรุปการจอง

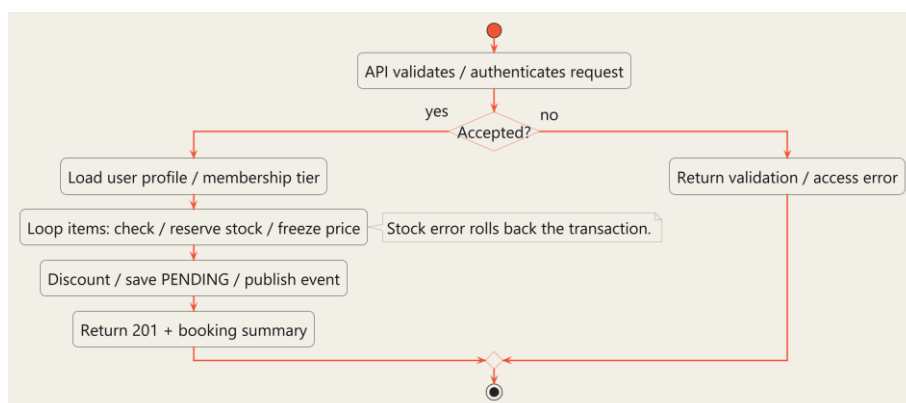


ภาพที่ 31 Sequence ฉบับสรุปการบันทึกเปิดซอง

ค Sequence และ Activity ฉบับสรุป



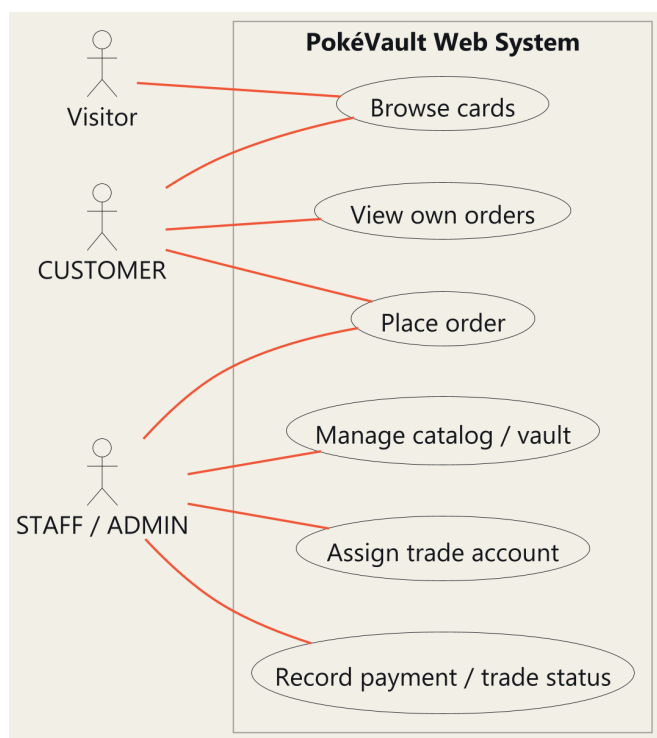
ภาพที่ 32 Sequence ฉบับสรุปการเทรด



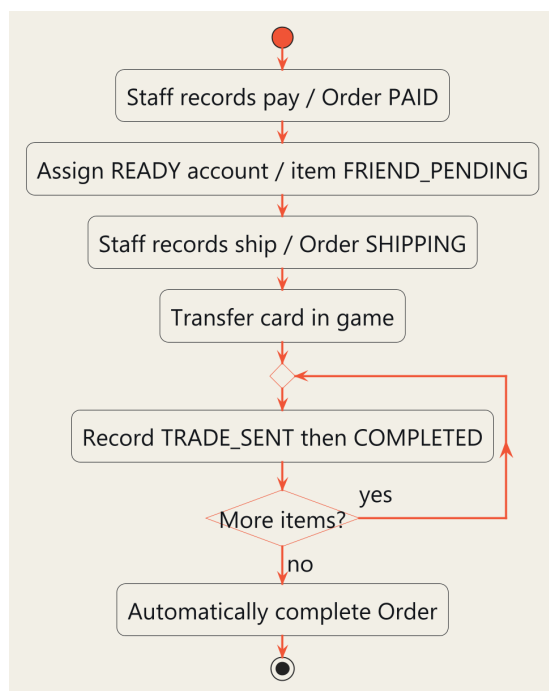
ภาพที่ 33 Activity ฉบับสรุปการจอง

ฉบับสรุปเลือก Message และ Collapse กิจกรรมบางส่วนเพื่อการนำเสนอ ไม่เพิ่มเมธอดที่ไม่มีใน Source และไม่ใช่แทนรายละเอียด Stock Loop หรือ Guard ของแต่ละ Item

ค Use Case และ Activity ฉบับสรุป

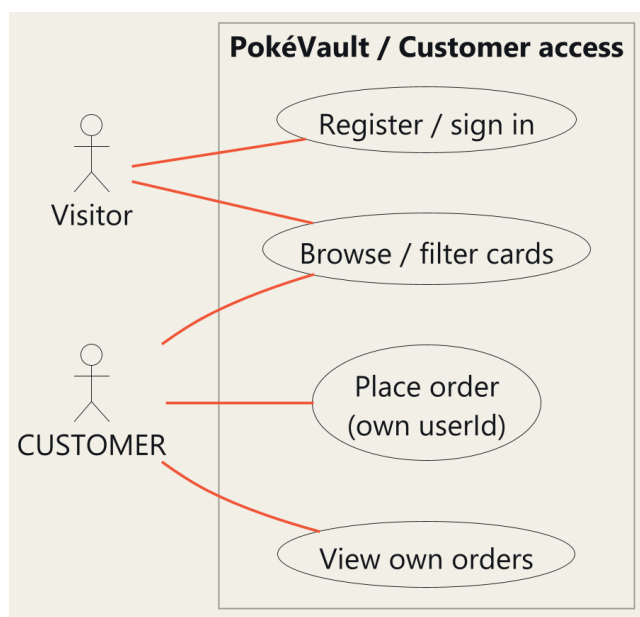


ภาพที่ 34 Use Case ฉบับสรุปของระบบ

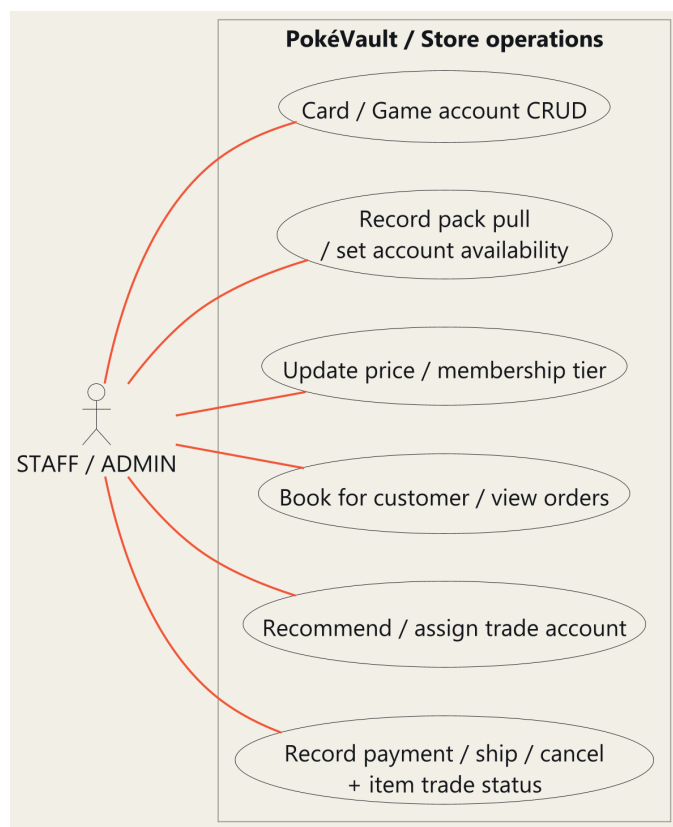


ภาพที่ 35 Activity ฉบับสรุปการส่งมอบ

ค Use Case ของลูกค้าและพนักงาน



ภาพที่ 36 Use Case ของ Visitor และ CUSTOMER



ภาพที่ 37 Use Case ของ STAFF และ ADMIN

ภาคผนวก ง การตรวจสอบตามหัวข้อประเมิน

ตารางที่ 39 แผนพื้นฐานตามหัวข้อประเมิน

หัวข้อ	หลักฐานในรายงาน	หลักฐานใน Repository
Diagram	บทที่ 1 ถึง 5 และ 7 พร้อมภาคผนวก ค	doc/diagrams/uml และ img/uml
Layered และ SOLID	บทที่ 2 พร้อมโค้ดและข้อจำกัด	doc/solid-analysis.md
Enterprise 6 และ GoF 3	บทที่ 3 และ Class Diagrams	doc/design-patterns.md
Database Design	บทที่ 5 และ Dictionary ภาคผนวก ก	schema.sql / db/manual
REST และ Swagger	บทที่ 6 พร้อมภาพ Production	API Controllers / OpenApiConfig
Testing	บทที่ 7 และภาคผนวก ข	test/java และ test/reports/project
Git README และโครงสร้าง	บทที่ 7	GitHub PR / .github / README
Deployment	บทที่ 7 และ URL สาธารณะ	Dockerfile / application.yml
ความเข้าใจโค้ด	คำอธิบาย Diagram เหตุผล และโค้ดแต่ละบท	Source ของสมาชิกและการตอบคำถาม